

ByteDance: Let bytes perform brilliantly in multi-view encrypted traffic classification

Yuwei Xu ^{id a,b,*}, Zhiyuan Liang ^{id a}, Xiaotian Fang ^{id a}, Kehui Song ^{id c}, Meng Wang ^{id d},
Qiao Xiang ^{id e}, Guang Cheng ^{id a,b}

^a School of Cyber Science and Engineering, Southeast University, Nanjing, 211189, China

^b Purple Mountain Laboratories, Nanjing, 211111, China

^c School of Software, Tiangong University, Tianjin, 300387, China

^d NSFOCUS Technologies Group Co., Ltd., Beijing, 100089, China

^e School of Informatics, Xiamen University, Xiamen, 361005, China

ARTICLE INFO

Keywords:

Encrypted traffic classification
Multi-view learning
Accurate byte feature extraction
Suppression mitigation for byte view

ABSTRACT

The use of deep learning in encrypted traffic classification has led to two primary methodologies: the time-series view (T-view) and the byte view (B-view). T-view ETC schemes capture timing relationships in network flow, whereas B-view ETC schemes focus on byte differences in raw traffic data. Recent studies show that using multi-view learning can improve ETC performance by creating technical complementarity between T-view and B-view. However, the existing multi-view studies have failed to achieve significant improvements over single-view schemes. Through an in-depth analysis of comparative experiments, we identify two major problems. Byte features in raw traffic are not accurately extracted, and the B-view is suppressed by the T-view during model training. To tackle the problems highlighted above, we propose ByteDance, which lets bytes play brilliantly in multi-view ETC. To accurately capture byte features, a two-stage byte feature extraction strategy (TBFE) is proposed. It screens packet bytes by their protocol format and distribution, then employs a local attention module and a transformer encoder to extract byte features accurately. To mitigate the suppression of B-view, a prototype-network-based dynamic gradient compensation strategy (PDGC) is designed. PDGC assigns weights to gradients in each training batch, enhancing gradient propagation in the B-view with a PCE loss function. We compare ByteDance with nine state-of-the-art ETC schemes. Experimental results show that ByteDance's classification accuracy on four datasets significantly exceeds that of the suboptimal schemes. Moreover, ByteDance leads the pack with its impressive metrics, including parameter counts, GPU memory demands, and both training and inference times for models, showcasing remarkable operating efficiency.

1. Introduction

As the Internet becomes essential in our lives, users worldwide are increasingly focused on protecting their personal privacy while enjoying its benefits. Network applications increasingly use cryptography to authenticate identities and protect the confidentiality of transmitted data. According to Google statistics, as of Feb. 2025, the proportion of encrypted traffic on the Internet has reached 95% [1]. Accurate traffic classification is crucial for resource optimization in network management [2] and for preventing malicious attacks in network security [3]. The transition from plain text to encrypted communication has significantly hindered academic researchers and industry professionals in their efforts to accurately detect network traffic.

Recent breakthroughs in deep learning (DL) have revolutionized the fields of computer vision (CV) [4] and natural language processing (NLP) [5], sparking a wave of research into its applications for encrypted traffic classification (ETC). These efforts have yielded impressive outcomes, demonstrating the effectiveness of DL in this area. Current research in DL-based ETC has primarily adopted methodologies from CV and NLP to analyze and classify encrypted traffic from different views [6]. The existing studies are divided into two technical routes: time-series view (T-view) and byte view (B-view). T-view ETC analyzes the network flow by treating it as a series of packets, creating a time-series that captures features such as packet lengths [7], directions [8–10], and timestamps [11]. This time-series is then used as input for classification in a deep learning model. B-view ETC takes the transmitted data

* Corresponding author.

E-mail addresses: xuyw@seu.edu.cn (Y. Xu), liangzy@seu.edu.cn (Z. Liang), fangxt@seu.edu.cn (X. Fang), songkehui@tiangong.edu.cn (K. Song), wangmeng7@nsfocus.com (M. Wang), qiaoxiang@xmu.edu.cn (Q. Xiang), chengguang@seu.edu.cn (G. Cheng).

<https://doi.org/10.1016/j.comnet.2025.111843>

Received 24 February 2025; Received in revised form 14 September 2025; Accepted 3 November 2025

Available online 8 November 2025

1389-1286/© 2025 Elsevier B.V. All rights reserved, including those for text and data mining, AI training, and similar technologies.

bytes, restructures them into text [12–14] or an image [15–18], and inputs them into a specific deep learning classification model. The above two routes have their own advantages and disadvantages. T-view ETC effectively captures global packet distribution characteristics but often overlooks significant local traffic variations. B-view ETC analyzes the discrepancies in specific bytes of traffic data from a localized standpoint. While the model accounts for correlations between various locations, its view is frequently constrained by an overemphasis on local positions.

In order to achieve technical complementarity, some researchers have proposed multi-view approaches [19–26]. The raw traffic is analyzed from several views, and customized neural networks are employed to extract specific features. Ultimately, these features are combined to facilitate accurate traffic classification. Currently, several researchers have put forward multi-view ETC schemes aimed at tackling complicated tasks such as malicious network traffic detection [6], Tor website fingerprinting attacks [27], and TLS traffic classification [28]. However, recent research efforts are grappling with a considerable challenge. Unexpectedly, multi-view schemes have not significantly improved traffic classification performance over single-view schemes. In some tasks, their accuracies are even slightly lower than that of a single-view solution [29,30]. Some researchers thought that the models performed poorly because they did not effectively fuse features from different views. They tried to propose some improved solutions, but the effects were not significant.

To explore the dilemma of multi-view ETC, we carried out extensive comparative experiments. The findings reveal that, unlike T-view, the potential of B-view remains underutilized within the model. There are two main reasons: (1) **The byte features of the raw traffic are not accurately extracted.** Firstly, many existing schemes use the bytes of the application layer payload as input, but this data is often obscured and diffused by cryptographic techniques, which decreases the presence of useful features. Secondly, the majority of current research organizes bytes into either a one-dimensional text or a two-dimensional image, subsequently tailoring neural networks to extract relevant features. This processing scheme prioritizes local byte features, often overlooking the meaning of each field in packets and their logical connections, resulting in missed key features. (2) **During model training, B-view is easily suppressed by T-view and is difficult to play its due role.** Due to the substantial differences in data distribution and time-series dimensions between the features extracted from these two views, the gradients along their respective paths will vary in magnitude during the model's backpropagation process. The gradient descent optimization algorithm tends to prioritize updating parameters from T-view with larger gradients, while the B-view with smaller gradients often receives less optimization. The phenomenon of gradient competition ultimately results in B-view being significantly overshadowed by T-view in multi-view ETC. Existing research has concentrated on the design of feature extraction and fusion modules, but has overlooked the issue of B-view suppression in gradient in gradient optimization optimization.

To address the shortcomings of existing research, we propose a multi-view ETC scheme. By employing precise byte feature extraction and effective model gradient optimization, the full potential of B-view is realized in the tasks of classifying encrypted traffic. Thus, we decided to name this scheme ByteDance. The main contributions of this paper can be summarized in the following four aspects.

- To analyze performance bottlenecks in current multi-view ETC research, three popular schemes were tested on four typical encrypted traffic datasets. The experimental results reveal that there are two major problems in existing studies. First, blindly selecting bytes from encrypted packet payloads as input for B-view hinders the model's ability to extract meaningful byte features, which decreases classification performance. Second, the huge gap in gradient descent on the backpropagation path will cause T-view to suppress the B-view.
- To accurately capture the byte features in the traffic, a two-stage byte feature extraction strategy (TBFE) is proposed. In the first stage, a

rough byte-screening process is conducted to eliminate the encrypted payload portion with low feature density. This is achieved by examining the format and byte distribution of each layer in the network protocol stack. In the second stage, we carry out precise extraction of byte features. The local attention module captures detailed byte features from each layer in the protocol stack, while the transformer encoder extracts global context information from the session.

- To mitigate the suppression of B-view by T-view, a prototype-network-based dynamic gradient compensation strategy (PDGC) is designed. In each training batch, our model calculates the Euclidean distance between the path feature vector and its prototype vector from different views. According to this calculation, the model dynamically assigns weights to the gradients of different paths. Furthermore, by integrating the PCE loss function, the model effectively amplifies the gradient propagation strength in the B-view, thereby optimizing the model's overall performance.
- To validate ByteDance, we conducted a comparison against nine cutting-edge ETC schemes using four real-world datasets of encrypted network traffic. Experimental results indicate that ByteDance achieved accuracy rates of 95.23 %, 96.57 %, 96.23 %, and 95.62 %, improving by 0.41 %, 0.35 %, 3.27 %, and 2.57 % over suboptimal schemes. Besides, operating efficiency experiments show that ByteDance has excellent efficiency. Finally, ablation experiments show that our TBFE and PDGC strategies play significant roles in classification performance.

The rest of this paper is organized as follows. In Section 2, we summarize previous related research, covering single-view ETC and multi-view ETC. In Section 3, we highlight two critical issues found in the existing multi-view ETC research through comparative experiments, which serve as the motivation for this work. Section 4 presents an in-depth overview of ByteDance's scheme design, detailing the strategies of both TBFE and PDGC. We introduce the experimental setup in Section 5 and evaluate ByteDance's classification performance and operating efficiency in Section 6. Finally, a conclusion is given in Section 7.

2. Related work

In this section, we first discuss the single-view ETC schemes. After reviewing T-View and B-View, we stress the potential of multi-view ETC research. Secondly, we provide an overview of existing multi-view schemes and introduce the dilemma currently faced by researchers. As shown in Table 1, we summarize the essential details of all relevant ETC schemes while emphasizing ByteDance's advantages.

2.1. Single-view ETC

Initial studies on ETC leveraged technical approaches from the field of artificial intelligence. Researchers characterized the raw encrypted traffic from a single view, ultimately developing a classification model to facilitate detection. The mainstream research work is divided into T-view ETC and B-view ETC.

The T-view ETC schemes treat network flow as a series of packets and create a time-series for the classification model by using features like packet length, direction, and timestamps. In [7], the authors proposed FS-NET, which uses the packet length sequence of sessions as input. FS-NET employs a Bidirectional Gated Recurrent Unit (BiGRU) [31] based encoder-decoder structure to extract features and incorporates a reconstruction mechanism to enhance feature representation. In [8], the authors presented Deep Fingerprinting (DF), which utilizes 1DCNN to extract features from Tor traffic, focusing on the direction of packets to achieve high accuracy in classifying both defended and undefended traffic. In [9], the authors introduced the RF method, which employs CNNs to aggregate and extract features from packet direction sequences, enabling efficient identification of Tor traffic. Similarly, in [10], the authors proposed WF-Transformer, which uses the direction

Table 1

A summary of existing encrypted traffic classification schemes.

Ref.	Type	Classifier	Input	Input Size	Field	NDP	AS
[7]	T	BiGRU	PLS	200 x 1	APP	✓	×
[8]	T	CNN	PDS	5000 x 1	TOR	✓	×
[9]	T	CNN	PDS	1800 x 2	TOR	✓	×
[10]	T	Self-attention	PDS	[1000–5000] x 1	TOR	✓	×
[11]	T	BiLSTM, Self-attention	PTS, PDS	–	WFA	✓	×
[12]	B	BERT	PB	256	M-APP/TOR/VPN/TLS/MT	×	×
[13]	B	BERT	PB	128	VPN	✓	×
[14]	B	Self-attention	FB	–	MT/VPN/TLS/IoT	×	×
[15]	B	1DCNN	SB	28 x 28	VPN	×	×
[16]	B	1DCNN, BiLSTM	FB	10 x 1500	MT/VPN/TOR	×	×
[17]	B	Attention-CNN	PB	1 x 50	PC/APP/VPN	×	×
[18]	B	Variant ResNet18	PB	32 x 32 x 3	VPN/TLS	✓	×
[19]	B&T	1DCNN, BiLSTM	SB, PLS	B: 1014 x 256, T: 20 x 128	M-APP	×	×
[20]	B&T	1DCNN, BiLSTM	FB, PB, PTS, PLS	B: 15 x 64, 1 x 500, T: 15 x 2	ID/IoT	×	×
[21]	B&T	Self-attention, BiLSTM	SB, PLS	B: 10 x 400, T: 10 x 1	TLS	×	×
[22]	B&T	PSAE, FGRU	SB, PLS	B: 20 x 40, T: 20 x 1	VPN/TOR	✓	×
[23]	B&T	Self-attention	PLS, PTS, PB	B: 16 x 1024, 4 x 713, T: 16 x 3	MT	×	×
[24]	B&T	CNN, Attention	SB, PLS, PDS, PTS, PWS	B: 1 x 784, T: 32 x 4	MT/VPN/TOR/IoT	×	×
[25]	B&T	CNN, GNN	Header PB, Payload PB	B: 15 x 350, T: 15 x 50	VPN/TOR	×	×
[26]	B&T	1DCNN, GRU	SB, PLS, PDS, PTS, PWS	B: 24 x 24, T: 12 x 4	M-APP	×	✓
ByteDance	B&T	Attention	SB, PLS, PDS	B: $N \times M$, T: $N \times 1$, $N \times 1$	TLS/VPN/TOR/M-APP	✓	✓

In the header, “NDP” represents that the scheme does not depend on application-layer payloads. “AS” indicates that the scheme is capable of alleviating the suppression that arises among different views. In the type column, “B” denotes B-View and “T” indicates T-View. In the input column, “PLS” denotes the packet length sequence, “PDS” refers to the packet direction sequence, “PTS” identifies the packet timestamp sequence, “PWS” indicates the packet TCP window size sequence, “PB” stands for packet bytes, “FB” signifies flow bytes, and “SB” represents session bytes. Input size represents the shape of the represented traffic data before it is input into the model. For ByteDance, N refers to the first N packets of a bidirectional flow, and M refers to the first M bytes of each packet. The field column outlines the various application scenarios for each ETC scheme. “M-APP” denotes a range of mobile applications, while “IoT” encompasses the numerous applications of the Internet of Things. “MT” signifies malicious traffic detection, “ID” refers to intrusion detection, and “WFA” represents the diverse types of website fingerprinting attacks. “VPN” indicates various VPN tools, while “TOR” means the most popular anonymous network.

sequence of packets in Tor traffic as input and employs a transformer [32] network to learn temporal features, boosting Tor encrypted traffic classification by capturing long-range dependencies. In [11], the authors introduced an input-agnostic deep learning framework that incorporates packet timestamp and direction to create packet vectors, which are then processed through a hybrid neural network composed of Bidirectional Long Short-Term Memory (BiLSTM) and attention to learn temporal features, enhancing traffic classification by capturing correlations between packets and flows. T-view ETC schemes excel in effectively capturing the overall distribution characteristics of packets within a flow. However, they often overlook the critical variations in local packet fields across different types of encrypted traffic. Moreover, these schemes typically necessitate the capture of the complete network flow as input and tend to underperform in classification tasks that only capture partial traffic fragments.

The B-view ETC schemes are inspired by research in the fields of NLP and CV. It converts raw bytes of encrypted traffic into text or image form, then uses a deep learning model for classification. In [12], the authors treated the bytes in network traffic as text and developed ET-BERT, drawing inspiration from the traditional BERT model [33] used in NLP. ET-BERT pre-trains deep contextualized datagram-level representations from the raw bytes of large-scale unlabeled traffic and achieves state-of-the-art performance on five ETC tasks by fine-tuning on small amounts of labeled data. The authors of [13] proposed a BERT-based scheme that processes encrypted traffic data using packet header information and then convert header field values into 2-byte tokens to form sentences for BERT’s pre-training and fine-tuning, enhancing classification accuracy and generalization performance. In [14], the authors proposed EAPT, an ETC model using adversarial pre-trained transformers, which tokenizes encrypted traffic data with SentencePiece to address coarse tokenization granularity. EAPT achieves high performance across multiple datasets with fewer model parameters. Given the remarkable advancements achieved by convolutional neural networks (CNNs) in CV, some researchers have attempted to transform raw traffic bytes into “im-

ages” for enhanced model classification. In [15], the authors proposed an end-to-end approach that converts the first 784 bytes of flows into grayscale images and inputs them into a 1DCNN for classification. Similarly, the authors of [16] utilized the first 1500 bytes of the first 15 packets in a session to generate a two-dimensional image and extracted spatiotemporal features using 1DCNN and BiLSTM sequentially. In [17], the authors presented a self-attentive deep learning approach that processes traffic data as images, using packet-level input and CNN to learn features, while the self-attention mechanism highlights important input parts, boosting classification accuracy and interpretability. The authors of [18] designed EETC, an ETC scheme based on a variant ResNet network and incremental learning. It preprocesses traffic data into RGB images and supports continuous model updates without retraining from scratch, demonstrating effective classification of encrypted traffic. The B-view ETC schemes analyze raw bytes, allowing for an in-depth exploration of the local characteristics of traffic. Although some models take into account the correlation between features across various locations, their emphasis on local byte differences often restricts their overall perspective.

2.2. Multi-view ETC

To address the shortcomings of single-view ETC research and benefit from different views, some researchers have suggested using a multi-view deep learning model in ETC. This methodology entails analyzing traffic from various views, employing distinct neural network architectures to capture relevant features, and subsequently integrating all extracted features to enhance traffic classification.

In [19], the authors proposed a multi-view model named APP-NET. It employs 1DCNN to extract B-view features and LSTM to extract T-view features and then concatenates them for classification. In [20], the authors proposed MFFusion, which uses LSTM and CNN modules to extract features for the T-view and B-view, respectively, and then concatenates these views into different matrices. The authors of [21]

presented PEAN, an advanced multi-view model that integrates self-attention mechanisms with LSTM for the first time. The self-attention module extracts features from the B-view, while the LSTM module captures features from the T-view. In [22], the authors proposed DM-HNN, a multi-view model that utilizes SAE and GRU for feature extraction. It processes the byte features from the first 20 packets of a session through the SAE module and extracts the time-series features using the GRU module. In [23], a multi-view learning scheme based on byte features, PETNet, is proposed, which represents an advanced multi-view scheme. It utilizes a meta feature extractor and a transformer encoder to perform feature extraction on the TLS handshake packets and data packet byte sequences, respectively. Finally, the two features extracted from the T-view and B-view are directly concatenated and fused.

It is disappointing that the multi-view ETC schemes mentioned above did not significantly outperform the single-view solutions in ETC tasks. Researchers have noted that current approaches merely combine features extracted from various views without thoroughly integrating them before classification. This lack of deep fusion limits the potential for achieving outstanding performance [6]. According to the analysis, the authors of [24] proposed GLADS, which uses an “indicator” mechanism to handle B-view and T-view input, then employs a global-local attention layer for multi-view fusion. The fusion layer can extract timing and global information from the token output by the GLADS’s exchange layer. In [25], the authors introduced DE-GNN, which utilizes a dual embedding layer to separately process packet headers and payloads, followed by PacketCNN and GNNs for feature extraction, and an adaptive deep feature fusion (ADFF) mechanism is employed to fuse the features from both views. The research discussed above has paved the way for integrating fusion features in multi-view ETC. Despite their efforts resulting in some progress, they have yet to achieve a substantial enhancement in the classification model’s performance.

In addition, some researchers believe that there is a suppression problem between different views. The authors of [26] proposed MIMETIC, a method that pre-trains feature extraction modules independently and then freezes the parameters to avoid interference between T-view and B-view. The training model strategy effectively mitigates the suppression between views, leading to a marked improvement in the classification performance of mobile encrypted traffic. In conclusion, network encrypted traffic is characterized by both time-series features and byte features. Leveraging the potential of multi-view approaches to enhance the accuracy of ETC presents a significant challenge in today’s research landscape.

3. Problem analysis and research motivation

To investigate the performance bottleneck of multi-view learning in ETC, we have selected three cutting-edge schemes and performed comparative experiments across four encrypted traffic datasets. Based on experimental results, we analyze two major problems in the current research dilemma and propose ByteDance’s research motivation.

3.1. Inaccurate byte feature extraction

According to the discussion in Section 2.2, we selected three multi-view ETC schemes, APP-NET [19], PEAN [21], and DM-HNN [22], as research objects. In order to verify the effectiveness of feature extraction from each view, we have conducted a comparative experiment. For the three schemes, we set three configurations for extracting features from encrypted traffic: utilizing the T-view exclusively, the B-view exclusively, and a combination of both views. Fig. 1 shows their classification accuracy on different datasets. The results demonstrate that the accuracy achieved with T-view alone closely resembles that of using multiple views. If only the B-view is used, the accuracy is much lower than that of the other two configurations. The results indicate that these schemes failed to effectively extract features in the B-view, ultimately

compromising the overall performance of the multi-view learning model. There are two reasons for this problem.

First, most current schemes extract byte features from the application-layer payloads of packets. Nonetheless, the majority of application layer protocols opt to encrypt the data being transmitted, such as TLS 1.3 and QUIC. Once cryptographic technology obscures and diffuses the data within the payload, it becomes challenging for the deep learning model to extract meaningful features from these bytes. To verify this reason, we conducted a comparative experiment on the TLS dataset. We monitored the variations in the accuracy of the three schemes as we progressively increased the number of bytes truncated in the B-view. As shown in Fig. 2, when the number of bytes exceeds 60, the accuracy of these schemes continues to decline. This outcome aligns with the findings presented in FastTraffic [34].

Second, the existing schemes for extracting byte features from raw traffic have design flaws. These schemes typically organize bytes into one-dimensional text or two-dimensional images, then use classic neural network architectures from NLP and CV to extract features. However, this approach prioritizes local byte features, often overlooking the meaning of each field in packets and their logical connections, resulting in missed key features.

3.2. Suppression of B-view

As shown in Fig. 1, the introduction of B-view to the three multi-view ETC schemes does not obtain a significant enhancement in accuracy when compared to the scenario where only T-view is utilized for feature extraction. On the TLS1.3 and TROJAN-VPN datasets, the multi-view model even underperformed compared to the model that utilized a T-view. Inspired by [29,30], we further explored the impact of inter-view suppression on model classification performance. We conducted experiments to compare the effects of T-view and B-view, both used alone and adopted in a multi-view ETC model. To evaluate the model’s classification performance with a single view, we use the selected view for training in the three multi-view schemes. To assess a specific view in the multi-view model, we use module parameter replacement for evaluation. The test process consists of three steps: (1) train a multi-view model until it converges and extract parameters from the feature extraction module in the specific view; (2) load these parameters into a new model that focuses on this view, freeze the parameters, and train the classification module until model convergence; (3) use the new model to classify traffic and assess its performance. Fig. 3 depicts the changes in the accuracy of T-view and B-view as the number of model iterations increases. In the chart, the solid line illustrates the performance of the model trained with a single view, while the dotted line reflects the performance of the corresponding view when utilizing multi-view model (MM) training.

Fig. 3 shows that T-view is minimally suppressed during the multi-view model training, whereas B-view experiences varying levels of suppression across all three schemes. The phenomenon occurs because the gradient descent optimization algorithm tends to prioritize updating parameters from the T-view that exhibit larger gradients. The small gradient descent along the B-view path indicates that the multi-view model hasn’t fully optimized this path upon convergence. Therefore, additional training is necessary to enhance its performance. Most current multi-view ETC research overlooks the suppression problem between views, leading to limited improvement after feature fusion. In [26], the authors suggested first pre-training the feature extraction module for each view separately, then freezing the parameters for joint fine-tuning. Although this approach can somewhat mitigate the suppression of views, it necessitates the establishment of an intricate pre-training and fine-tuning framework, significantly increasing the complexity of model training. Existing studies in the field of artificial intelligence have demonstrated that multi-task training can be balanced by modulating gradient magnitudes and directions [35,36], offering substantial insights into mitigating the suppression of B-view.

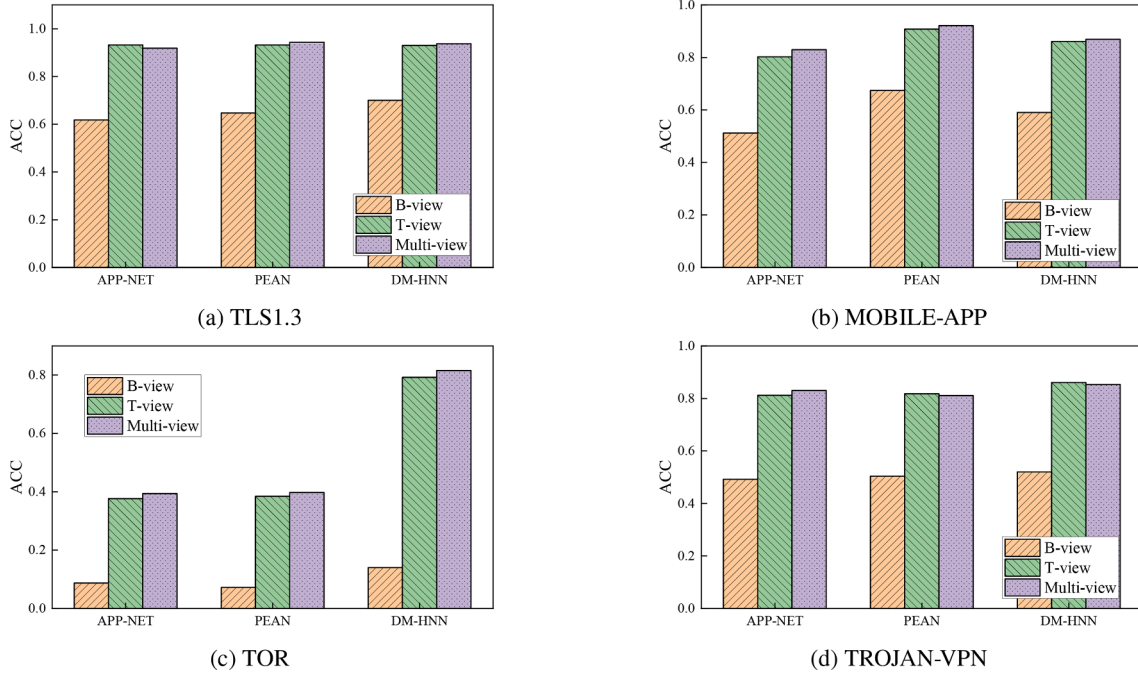


Fig. 1. Accuracy comparison among three configurations of three multi-view ETC schemes.

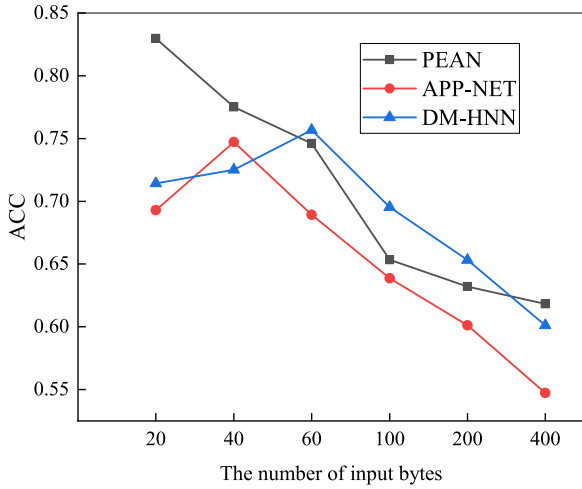


Fig. 2. Accuracy change of three schemes using only B-view with the increasing number of input bytes.

3.3. Motivation

In summary, existing multi-view ETC schemes struggle with two main problems: inaccurate byte feature extraction and suppression of the B-view. They restrict the potential of bytes to improve traffic classification performance in multi-view approaches. To solve the above problems, we propose ByteDance, which lets bytes play a full role in multi-view ETC. To accurately capture byte features, a two-stage byte feature extraction strategy (TBFE) is proposed. It first screens packet bytes based on their protocol format and byte distribution, then it uses the local attention module and transformer encoder to accurately extract byte features. To mitigate the suppression of B-view, a prototype-network-based dynamic gradient compensation strategy (PDGC) is designed. In every training batch, PDGC allocates weights to the gradients of multiple paths, enhancing the strength of gradient propagation in the B-view using a PCE loss function.

4. Methodology

In this section, we first introduce the overall structure of ByteDance, then give a detailed explanation of the core components.

4.1. Overall structure of ByteDance

The structure of ByteDance is shown in Fig. 4, which consists of a three-tier processing framework: the input layer, the feature extraction layer, and the fusion-balance layer. The raw session samples first undergo feature generation and data cleaning in the input layer. After the first stage of fuzzy byte selection processing by TBFE, a high-density byte matrix with high effectiveness is generated. The feature extraction layer, through a dual-view deep learning model, captures both time-series patterns and byte correlations, outputting high-dimensional feature vectors and multi-view representation tokens. The fusion-balance layer aligns heterogeneous feature spaces via tensor alignment, implements multi-source information fusion with a dynamic gating mechanism, and applies the PDGC strategy based on view representation tokens to compensate for the gradient of weak views, effectively addressing the under-optimization issue of B-view during joint training. The following parts will briefly introduce these three layers.

The input layer extracts three basic features from the raw session and performs a series of processing, including feature cleaning, as shown in Fig. 5. For T-view, the layer extracts a directional packet length sequence $\in \mathbb{R}^{1 \times N}$ to represent time-series features, followed by removal of zero-payload packets to eliminate invalid transmission interference. For B-view, the layer extracts a raw byte matrix $\in \mathbb{R}^{N \times MTU}$ from multiple packets to represent byte features. However, the matrix has low information density due to its inclusion of all bytes in the packet's Maximum Transmission Unit (MTU). The byte matrix is then processed to remove device fingerprint information (e.g., IP addresses and port numbers), mitigating biases from non-traffic-related features. Finally, the first stage of TBFE extracts a submatrix $\in \mathbb{R}^{N \times M}$ from the byte matrix based on the summarized byte distribution, transitioning from a low-density matrix to a high-density matrix with more effective information.

The feature extraction layer uses a dual-view model for multi-view learning. The T-view is built on the standard transformer encoder,

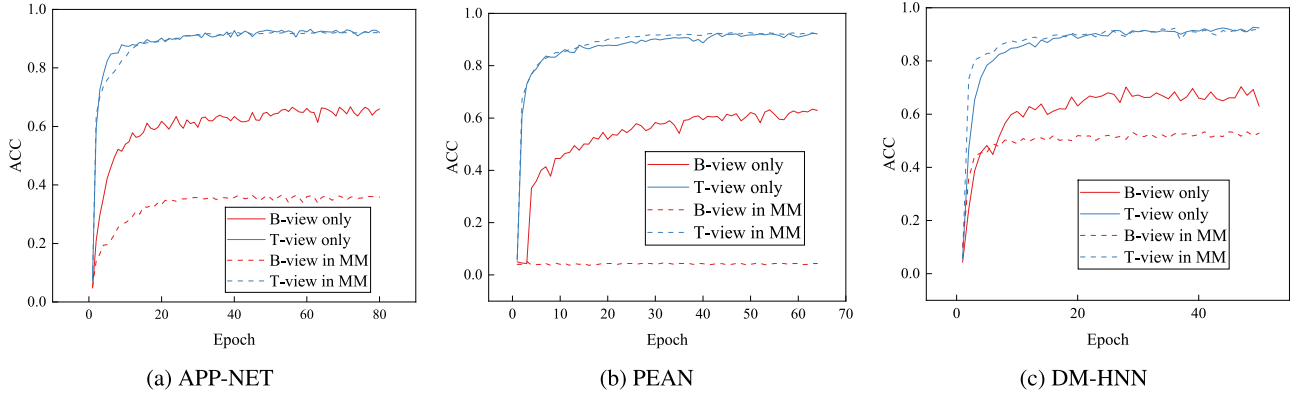


Fig. 3. Suppression of B-view and T-view in three multi-view ETC schemes.

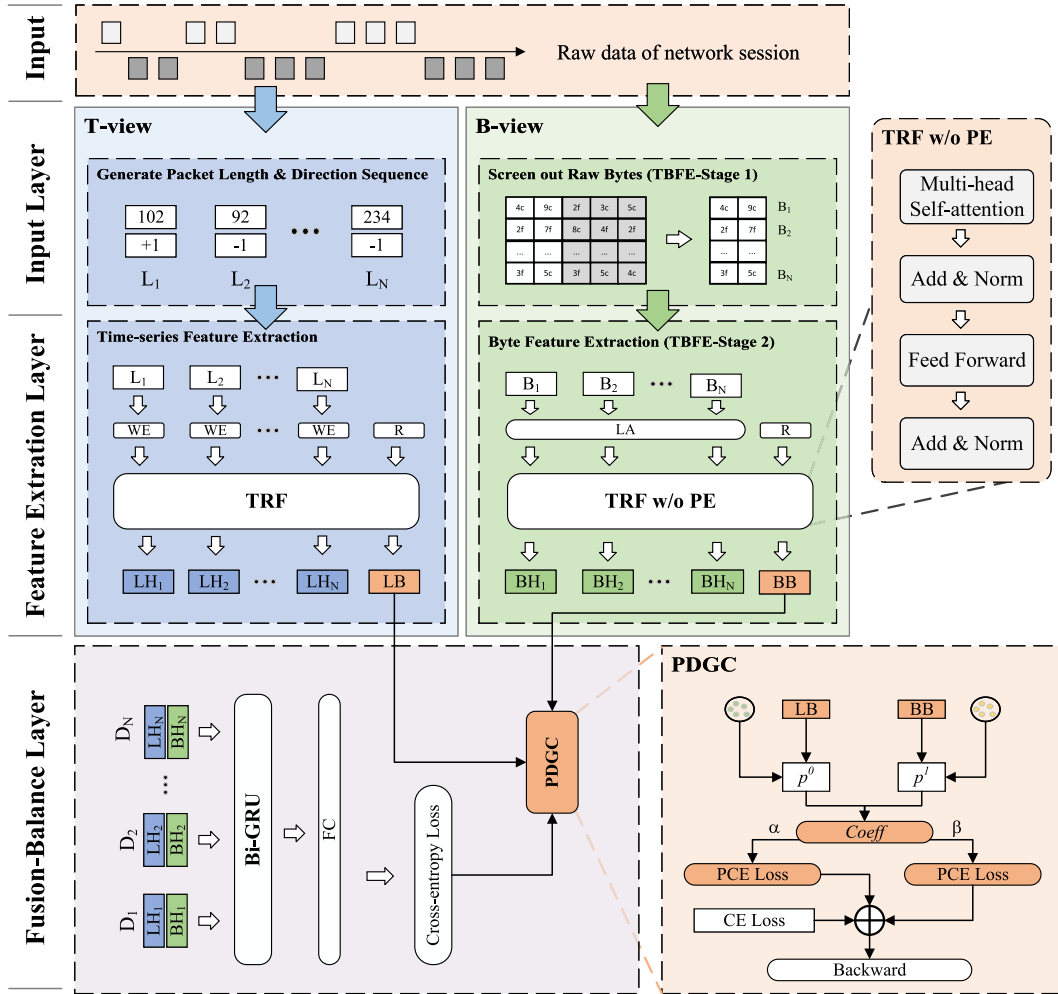


Fig. 4. The overall structure of ByteDance.

modeling the embedded time-series matrix to accurately capture the time-series patterns inherent in session behaviors. The second stage of precise byte extraction in TBFE is implemented in the B-view by combining a local attention (marked as “LA” in Fig. 4) mechanism with a global transformer encoder without positional embedding (marked as “TRF w/o PE” in Fig. 4), forming a hierarchical feature selection model. The local attention module focuses on byte-level semantic correlations within a single packet, while the cross-packet transformer layer establishes global dependencies. Ultimately, it outputs high-dimensional feature vectors with corresponding view representation tokens.

The fusion-balance layer consists of two parts: the feature fusion module and the PDGC module. In the fusion module, dual-view feature vectors are aligned at the packet level based on network session time continuity, then a time-series model performs deep fusion of the aligned heterogeneous feature sequences, generating fusion feature vectors with cross-view correlations, which are input into a cross-entropy loss function to compute the initial gradient. The PDGC module uses the two view representation tokens as gradient quantification bases, combined with PCE loss to construct a dynamic compensation mechanism. It focuses on gradient reweighting for the suppressed B-view, thereby

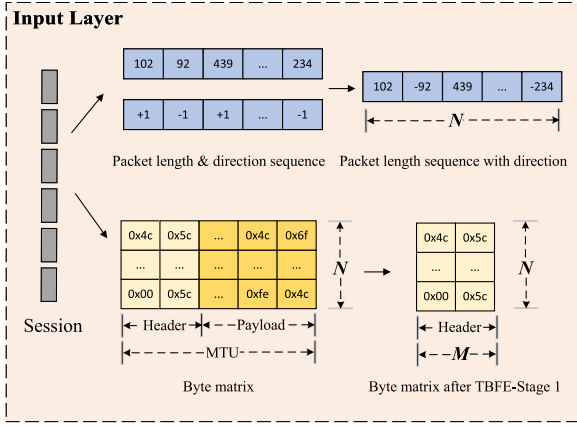


Fig. 5. The input layer of ByteDance.

Table 2
Details of network communication protocols.

Type	Protocol	NLH	TLH	ALH	M
Regular	TLS1.2/1.3	20	20	5+	45
	VMess	20	20	0	40
Proxy	VLESS	20	20	22+	62
	TROJAN	20	20	5+	45
	mKCP/KCP	20	8	24	52
Tor	TLS1.2/1.3	20	20	5+	45

maintaining the advantage of the T-view while effectively boosting the gradient contribution of the B-view.

4.2. Byte feature extraction process

The core of the extraction of byte features lies in the TBFE strategy, which includes the first stage of fuzzy byte selection and the second stage of precise byte extraction. The following provides a detailed description of these two stages.

The fuzzy byte selection occurs in ByteDance's input layer, as illustrated in Figs. 4 and 5. At this stage, only valid protocol header bytes will be kept, while invalid user payload data will be removed from the raw byte matrix. Given the raw byte matrix $\in \mathbb{R}^{N \times MTU}$ and valid byte count M for each traffic type, a submatrix $\in \mathbb{R}^{N \times M}$ is extracted, excluding user payload data. Here, N is a hyperparameter that defines the number of packets used for each traffic sample.

In this paper, we present the byte counts of common network encrypted traffic headers based on official protocol documentation and Wireshark [37] packet analysis, as shown in Table 2. The network layer header (NLH) length and transport layer header (TLH) length are relatively fixed, while the plaintext byte length in the application layer header (ALH) varies depending on the interaction phases between the client and server for the corresponding traffic type. For example, in TLS traffic, the plaintext information available in the application layer header during the data transmission phase consists of the first 5 bytes (content type, version, and payload length), whereas packets in the handshake phase contain additional plaintext information (e.g., server name indication). However, since handshake packets are relatively infrequent and lack general applicability, we define the available plaintext bytes in the application layer for TLS traffic in the table as "5+", representing the general case. The "+" symbol indicates that the application layer length may vary across different packet types in this class of encrypted traffic or may include optional fields. For generality, we only consider the byte count of the fixed and common plaintext fields. It is important to note that the selection of M does not require precise determination of minor byte differences, as the selection process in this stage is inherently fuzzy. In the first stage of the TBFE strategy, we apply fuzzy

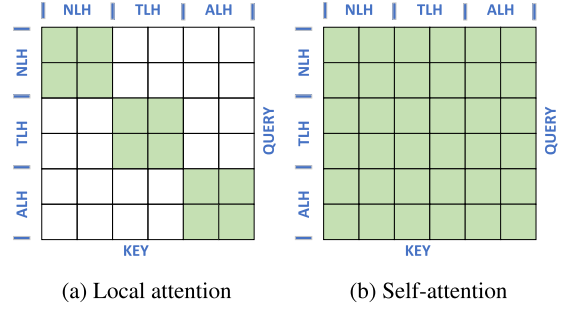


Fig. 6. Comparison of fixed-window local attention and self-attention.

truncation at the packet level to eliminate a large number of invalid byte sequences, ensuring that only meaningful bytes are retained.

The precise byte feature extraction occurs in the B-view path of the feature extraction layer. The core function is to extract accurate byte features from the byte matrix after fuzzy byte selection. This stage consists of two modules: the local byte feature extraction module and the global byte feature extraction module. As shown in Fig. 4, the former is marked as "LA", and the latter is marked as "TRF w/o PE".

The local byte feature extraction module is responsible for extracting features from the byte sequences within each packet. It employs a shared fixed-window local attention mechanism that is applicable across N packets. In contrast to conventional self-attention, fixed-window local attention is more favorable because it maintains the advantages of attention mechanisms while enabling the independent handling of packet headers within the byte sequence. This design fully exploits the inherent independence of different layers in the protocol stack. Fig. 6 shows that NLH, TLH, and ALH represent three separate attention windows within the module, each with a size based on the byte count in Table 2. Attention scores are calculated by multiplying each header's KEY with its QUERY, while avoiding misaligned headers to keep these headers independent. This design reduces interference from irrelevant information and enhances computational efficiency. Using a fixed-window local attention module shared across packets allows each packet's feature vector to effectively gather contextual information from nearby packets, enhancing computational efficiency.

Given the input byte matrix $B_{\text{input}} = \{B_1, B_2, \dots, B_N\}$, where B_i represents the byte sequence of the i -th packet with shape $\mathbb{R}^{1 \times M}$, we first obtain the matrices Q , K , and V via linear transformations. Next, attention scores are computed within the local range, normalized with Softmax, and the weighted sum of V is computed. The specific formulas are:

$$Q_i = B_{\text{input}} W_Q, K_i = B_{\text{input}} W_K, V_i = B_{\text{input}} W_V \quad (1)$$

$$\alpha_{ij} = \text{Softmax}\left(\frac{Q_i \cdot K_j^T}{\sqrt{d}}\right), \quad j \in \text{Local}(i) \quad (2)$$

$$o_i = \sum_{j \in \text{Local}(i)} \alpha_{ij} V_j \quad (3)$$

where W_Q , W_K , and W_V are the weight matrices, $\text{Local}(i)$ denotes the set of indices of the local Keys corresponding to the i -th Query, d is the dimensionality of each head, and α_{ij} represents the normalized attention weight, indicating the influence of j on i . $o_i \in \mathbb{R}^{1 \times r}$ represents the local attention output vector for the i -th element, which equals the weighted sum within the local range.

The global byte feature extraction module extracts features across byte sequences from multiple packets. It is implemented using a transformer encoder without positional embeddings. The self-attention mechanism within the transformer highlights the importance of each packet's byte vector while incorporating information from other packets, effectively capturing global byte-level patterns. The removal of positional

embeddings is intentional, as the shared-weight local attention used in the local byte feature extraction module allows the feature vectors of individual packets to learn information from other packets in advance. Consequently, the feature vectors of different packets lack a clear temporal relationship.

The transformer encoder used in this module comprises multi-head self-attention, residual connections, layer normalization, and a feed-forward network. Below is a brief explanation of the core multi-head self-attention mechanism.

The multi-head self-attention module consists of multiple self-attention heads. The core function of self-attention is to map a single input vector x (a packet byte vector) to a feature vector y that integrates information from multiple input vectors (other packet byte vectors). The input sequence to the transformer encoder is $O = \{o_1, o_2, o_3, \dots, o_N, R\}$, where o_i is the byte vector of the i -th packet, with shape $\mathbb{R}^{1 \times \nu}$. The sequence is primarily formed from the output of the local byte feature extraction module, with the final element R being a randomly initialized view representation token that aggregates features from other vectors, representing the path features corresponding to the view. This token serves as input to the subsequent PDGC module.

The Scaled Dot-Product Attention computes the attention scores for each head as follows:

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i) = \text{Softmax}\left(\frac{Q_i K_i^T}{\sqrt{d}}\right) V_i \quad (4)$$

where Q , K , and V are obtained from Formula (1), and d is the dimensionality of each head. The multi-head self-attention module combines the results of multiple attention heads, which are computed in parallel, using the formula:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O \quad (5)$$

where h represents the number of attention heads, and W_O is the projection matrix. Each attention head has its own set of weight matrices, allowing it to learn different relationships in distinct representational subspaces in parallel, thereby more effectively capturing various correlations among multiple packet byte vectors. Finally, the feature matrix $F_1 = \{BH_1, BH_2, BH_3, \dots, BH_N, BB\}$ is obtained through local and global byte extraction.

4.3. Time series feature extraction process

As shown in Fig. 4, the T-view path extracts the deep timing feature from the directed packet length sequence. It primarily consists of two components: word embedding and the time-series feature extraction module. The following part of this subsection will provide a detailed explanation of each component.

Word embedding transforms one-dimensional time-series into a two-dimensional matrix. Compared to the lower-dimensional space of one-dimensional vectors, higher-dimensional spaces provide more learning capacity, enabling the model to capture deeper latent patterns among elements in the sequence. Specifically, word embedding involves embedding each element in the packet length sequence into a vector. These vectors collectively form a trainable lookup matrix $E \in \mathbb{R}^{\nu \times u}$, where ν and u represent the range and dimensionality of the embedded vector values, respectively. Given an embedding matrix E and a sequence containing N elements ($L_{\text{input}} = [L_1, L_2, \dots, L_N]$), each element L_i ($i \in [1, N]$) is converted into a vector e_i by looking up the corresponding row in E . This process results in an embedded sequence $[e_1, e_2, \dots, e_N]$.

Time series feature extraction module is designed primarily to extract time-series features from multiple embedded packet length matrix. To build this module, we utilize a single layer of the standard transformer encoder. The choice of transformer over other time-series models is motivated by its self-attention mechanism, which can redistribute weights across multiple packet length vectors. This effectively selects a

subset of the N packet length vectors to represent the traffic distribution. More importantly, when combined with positional embedding, it enhances the model's ability to capture timing information.

The transformer encoder used in this module differs from the one in Section 4.2 by the addition of positional embedding. Next, we will briefly explain the concept of positional embeddings.

The self-attention mechanism of the transformer is inherently position-independent, meaning it cannot directly capture the sequential relationships of elements in the series. Positional embedding addresses this limitation by introducing position-related information into the input features, enabling the model to understand the temporal order of elements in the sequence. Given an embedding matrix $L_{\text{word embedding}} = [e_1, e_2, \dots, e_N]$, the positional embedding process is as follows:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right) \quad (6)$$

$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{(2i+1)/d_{\text{model}}}}\right) \quad (7)$$

$$L_{\text{embedded}} = L_{\text{word embedding}} + \text{PE} \quad (8)$$

In these formulas, "10000" controls the frequency of positional changes, ensuring that the positional embeddings vary at an appropriate rate across dimensions to capture relative positional relationships. L_{embedded} represents the input matrix after applying positional embedding, while PE denotes the positional embedding matrix.

After applying positional embedding, the resulting matrix is concatenated with a randomly initialized view representation token. This augmented input is then passed into the multi-head self-attention module. Using the calculations from Eqs. (1), (4), and (5), the final output vectors $F_2 = \{LH_1, LH_2, LH_3, \dots, LH_N, LB\}$ are obtained.

4.4. Feature fusion and balancing process

The core function of this process is to integrate different feature vectors extracted from multiple views and balance the optimization gradients of different views based on their respective weights. Therefore, this process is divided into two parts: the fusion module and the PDGC module, which are explained in detail below.

The fusion module primarily includes two steps: alignment and fusion. First, feature vectors extracted from multiple paths are concatenated along the feature dimension at the packet level. Then, based on the inherent temporal order between packets, the BiGRU is applied for feature fusion. Finally, the fused vectors are passed through a fully connected layer for dimensionality reduction, and the resulting vectors are input into the cross-entropy loss to calculate gradients. Given multiple feature matrices extracted from different paths, denoted as F_1, F_2 , the alignment operation is performed as follows:

$$[D_1, D_2, \dots, D_N] = \text{Concat}(F_1, F_2, \text{dim} = \text{feature}) \quad (9)$$

where N represents the number of packets, and D_i is the aligned feature vector of the i -th packet. This operation merges the features from multiple paths into a single aligned feature vector for each packet while preserving the temporal order among N vectors. Subsequently, the aligned feature sequence is fused using a BiGRU layer, as described below:

$$\bar{h}_t = \text{GRU}_{\text{forward}}(D_t, \bar{h}_{t-1}) \quad (10)$$

$$\bar{h}_t = \text{GRU}_{\text{backward}}(D_t, \bar{h}_{t+1}) \quad (11)$$

where $\bar{h}_t \in \mathbb{R}^H$ is the forward hidden state at time step t , and $\bar{h}_{t-1}$ is the hidden state from the previous time step.

The forward and backward states are concatenated at each time step t , and the final feature vector Z is computed as the mean over all time steps:

$$Z = \frac{1}{T} \sum_{t=1}^T \text{Concat}(\bar{h}_t, \bar{h}_t) \quad (12)$$

where T represents the total number of time steps.

This process ensures that the fused feature vector Z captures both forward and backward dependencies across the packet sequence, effectively integrating temporal and spatial features for subsequent processing.

The PDGC module is designed to address the issue where the optimizer tends to favor view with larger gradients during optimization, leading to under-optimization of other view. Inspired by the work in [29,30], we investigated the manifestation of this imbalance problem in the field of ETC. As shown in Fig. 3, the primary issue is the suppression of B-view by T-view.

To address this issue, inspired by [30], this paper proposes the PDGC strategy and applies it to the field of ETC. This strategy utilizes a prototype network to efficiently learn the features of each path, generating a prototype vector to represent each path. During each iteration, the similarity between a path's feature vector and its corresponding prototype vector is computed. Based on this similarity, a weight is assigned to the path, and the ratio of weights between different feature paths is used to represent the gradient ratio. This ratio is then used to calculate the path's PCE loss coefficient. This process strengthens weaker paths by balancing their optimization. Next, we will briefly introduce the concepts of prototype vectors, path weight, and PCE loss.

Prototype vectors are used to represent the central distribution of features for each category and are typically obtained by averaging the feature vectors of samples within the category. For a given category k , with its sample feature set $\{z_{ki}\}_{i=1}^{W_k}$, the prototype vector is defined as:

$$c_k^j = \frac{1}{W_k} \sum_{i=1}^{W_k} z_{ki}^j \quad (13)$$

where W_k denotes the number of samples in category k , z_{ki} represents the feature vector of the i -th sample, and c_k^j captures the overall distribution characteristics of category k in view j . In this work, there are two views in total, and the actual values of z_{ki} correspond to the view representation tokens of the respective categories.

At the beginning of each epoch, the prototype vectors for all path categories are computed. During each internal iteration, the softmax value representing the similarity between the feature vectors and their corresponding prototype vectors is calculated to determine the weight of each path. The calculation formula is as follows:

$$p_i^j(y = k | x_i^{v_j}) = \text{Softmax}\left(-d\left(\phi^j(x_i^{v_j}), c_k^j\right)\right) \quad (14)$$

where $x_i^{v_j}$ represents the input data, $\phi^j(x_i^{v_j})$ represents the $x_i^{v_j}$ after feature extraction, and j denotes the specific view under consideration. The term $-d()$ denotes the Euclidean distance, and the negative sign ensures that the smaller the distance, the larger the output value. This implies that the closer two vectors are, the more effectively the optimizer focuses on optimizing the corresponding path for that view. However, this effect can lead to the over-optimization of the favored path in a given view, resulting in the under-optimization of other paths. After obtaining the weights for each path, inspired by [30], the ratio of weights is used to represent the gradient ratio between the paths, as shown in the following formula:

$$\rho = \frac{\sum_{i \in B^0} p_i^0}{\sum_{i \in B^1} p_i^1} \quad (15)$$

where B^0 and B^1 represent the mini-batch training batches of the corresponding T-view and B-view, respectively.

After obtaining the gradient ratio, we use PCE loss to further train the weaker view. The PCE loss is a prototype-based loss function designed to improve the classification performance of the model by optimizing the distance between samples and their category prototypes. Its core idea is to use the previously calculated prototypes for each category and perform supervised training by minimizing the distance between the samples and the prototypes of the correct categories, while maintaining distinctiveness from the prototypes of other categories. The

formulation of PCE loss is similar to that of standard cross-entropy loss. However, while cross-entropy loss optimizes the discrepancy between the predicted class distribution and the true distribution, PCE loss models the similarity between the samples and the prototypes. Its specific definition is as follows:

$$L_{\text{PCE}}^j(f) = \mathbb{E}_{p(x_i^{v_j}, y)} \left[-\log \text{Softmax}\left(-d\left(z_i^{v_j}, c_y^j\right)\right) \right] \quad (16)$$

where $z_i^{v_j}$ is the representations of the sample x_i in view j , c_y^j is the prototype vectors (category centers) of category y in view j , which are obtained through the category's sample data distribution. In this paper, the PCE loss is computed for the two views (v_0 and v_1) independently, optimizing the feature representations under different views to enhance weaker views.

In summary, the loss function used in this paper is a combination of the cross-entropy loss and the PCE loss. The specific formula is as follows:

$$\mathcal{L}_{\text{acc}} = \mathcal{L}_{\text{CE}} + \alpha \cdot \mathcal{L}_{\text{PCE}}^0 + \beta \cdot \mathcal{L}_{\text{PCE}}^1 \quad (17)$$

$$\text{Coeff}(x) = 1 - \frac{1}{\log_2(x + 1)} \quad (18)$$

$$\begin{cases} \alpha = -\text{Coeff}(\rho), & \beta = 0, & \text{if } \rho \leq 1 \\ \alpha = 0, & \beta = \text{Coeff}(\rho), & \text{if } \rho > 1 \end{cases} \quad (19)$$

where ρ is gradient ratio, representing the T-view path weight relative to the B-view path weight. The α and β are the PCE loss coefficients, which are derived from the path weights calculated using Eq. (17)–(19). The Eq. (18) reflects a logarithmic growth characteristic, where the growth rate of the coefficients gradually slows as the ratio between paths increases. This design prevents excessively large PCE loss terms from overly influencing the gradient calculations of the raw cross-entropy loss function.

5. Experiment setup

In this section, we describe the experimental settings in detail, including datasets, experimental parameters, baseline schemes, and evaluation metrics.

5.1. Datasets

In our experiments, we evaluate ByteDance alongside nine baseline schemes utilizing four distinct datasets. TLS1.3 and Mobile-App are publicly available datasets, while TOR and Trojan-VPN are self-collected datasets. All of these data sets are collected from real-world networks, and their details are described in Table 3.

- **CSTNET-TLS1.3** [12] dataset contains TLS 1.3 traffic from 120 websites, captured in 2018. The collection lasts a week, collecting 15 minutes of data daily, totaling more than 100 GB of traffic data. For simplicity, we use TLS1.3 to represent it in the text. Due to the limited number of samples for certain websites in the original dataset, we carefully chose 50 categories that provide adequate sample sizes for the experiments.
- **MOBILE-APP** [38] dataset contains network packet traces from 2023, collected on the 5G infrastructure at Chalmers University of Technology. It includes 1912 pcap files. Each pcap file captures 1 minute of encrypted network traffic generated by one of eight popular mobile applications.
- **TOR** dataset was collected from July 12 to 20, 2023, using the Round-Robin method described in [8,39,40]. It includes traffic data from 25 commonly accessed websites in version 0.4.7.13 of the Tor service.

Table 3
Details of four network traffic datasets.

Dataset	Classes
CSTNET-TLS1.3	163, 51.la, 51cto, acm, adobe, alibaba, alipay, amap, apple, arxiv, baidu, bilibili, biligame, booking, chia, cloudflare, cloudfront, cnblogs, criteo, deepl, eastday, facebook, feishu, ggph, github, gmail, google, huanqiu, huawei, ibm, icloud, ieee, jd, msn, netflix, nike, office, overleaf, qq, sciencedirect, snapchat, sohu, t.co, thepaper, weibo, wikimedia, xiaomi, yahoo, youtube, zhihu
MOBILE-APP	facebook, instagram, linkedIn, spotify, tiktok, twitter, wikipedia, youtube
TOR	163, alibaba, aliyun, amazon, apple, baidu, bilibili, bing, cdc, csdn, deepl, epicgames, github, icloud, jd, msn, office, openai, qq, sci-encodedirect, stackoverflow, tencent, weibo, yahoo, zhihu
TROJAN-VPN	amazon, facebook, google, nytimes, reddit, twitter, wiki, youtube, weibo

Table 4
Training parameter selection settings and results.

Training Parameters	Search Range	Final
Training Epochs	[5, 100]	70
Batch Size	[32, 256]	128
Learning Rate	[1E-4, 1E-2]	1.5E-3
Optimizer	SGD Adam AdaGrad LambdaLR	Adam
Learning Rate Scheduler	ExponentialLR CosineAnnealingLR	ExponentialLR

- **TROJAN-VPN** dataset contains over 50GB of VPN traffic data from nine commonly used websites. From July 12, to 19, 2022, we employed Python scripts to simulate regular access proxy traffic using V2Ray [41] technology. Detailed information can be found in [42].

Detailed category information for the datasets is listed in Table 3. In order to address the issue of differing sample sizes across various categories, all categories within the dataset were uniformly downsampled to a consistent sample size. Besides, we adopted 5-fold cross-validation to ensure the objectivity and rigor of the experiments.

5.2. Experimental parameter settings

For all experiments conducted on the aforementioned four datasets, the parameters employed in ByteDance's model must be maintained consistently.

In T-view, the input packet length sequence is converted into vectors $\in \mathbb{R}^{1 \times N}$, where N is a hyperparameter indicating the number of packets per session sample. These are then transformed into matrices $\in \mathbb{R}^{N \times 128}$ through an embedding layer, followed by feature extraction using a transformer encoder. The multi-head attention in the transformer encoder has eight heads, an embedding dimension of 128, and a hidden layer dimension of 1024.

In B-view, the input data is in a dimension of $\mathbb{R}^{N \times M}$, where M is the valid byte count set in the TBFE's initial stage. The data is processed through a fixed-window local attention module, resulting in a matrix in $\in \mathbb{R}^{N \times 256}$, which serves as input for a transformer encoder layer like T-view. The local attention module has four attention heads, with a hidden layer dimension of 128 and an output dimension of 256. As outlined in Section 4.2, the window size is determined by the byte count available within the protocol stack of the corresponding session.

The training parameters are summarized in Table 4. A batch size of 128 is used, and all ETC models are trained with the Adam optimizer with a weight decay of $1E-3$. The starting learning rate is $1.5E-3$ and is gradually decayed using an ExponentialLR scheduler with a decay factor of 0.992. The training process is designed to last for a maximum of 70 epochs, but early stopping will occur if the validation accuracy fails to improve over 800 consecutive batches. All experiments are conducted on a host with an AMD Ryzen 7 5700X CPU, 32GB RAM, and one NVIDIA GeForce RTX 3090 GPU. The framework runs on Python 3.10, and the deep learning platform is PyTorch 1.12.1. Unless otherwise specified, all baseline models are trained with the same settings.

5.3. Baselines

We reproduce nine state-of-the-art ETC schemes. Among the baselines, FS-NET and RF are classical T-view methods, whereas ET-BERT and TSCRNN adopt a B-view perspective. The remaining five include advanced multi-view schemes such as MFFusion, PEAN, DM-HNN, PET-Net, and GLADS. The detailed information on these schemes is described as follows.

- **FS-NET** [7] is a T-view scheme that uses a packet length sequence of the first 200 packets in each network flow. It captures timing features from the sequence of packet lengths by employing BiGRU and creates a reconstruction mechanism to improve feature representation.
- **RF** [9] is a state-of-the-art method for launching web fingerprinting attacks on Tor traffic. By grouping nearby packets based on specific time slots, a 2D aggregation matrix is created for model input. Each element indicates the packet count in a particular direction for that time slot. It uses CNN for feature extraction and achieves the best performance on multiple datasets.
- **ET-BERT** [12] is an advanced B-view scheme that builds on BERT. The model is pre-trained on large-scale unlabeled data and fine-tuned on task-specific labeled data for classification. Given that packet-level ET-BERT consistently outperforms its flow-level counterpart, we adopt packet-level ET-BERT as the benchmark for comparison.
- **TSCRNN** [16] is a B-view scheme that utilizes the first 15 packets of each flow and restricts the length of each packet to 1500 bytes. It employs a two-layer 1DCNN coupled with a BiLSTM module for traffic classification.
- **MFFusion** [20] is a multi-view scheme that uses the first 64 bytes of the first 15 packets from each flow as flow byte features, and the first 500 bytes of each packet as packet byte features. Besides, it also extracts time-series features from the sequence of packet length and arrival interval time. Finally, MFFusion designs a network stacked with LSTM and CNN modules to fuse these features.
- **PEAN** [21] is an excellent multi-view scheme in recent research. It takes the first 400 bytes of the first 10 packets as B-view input and the packet length sequence from the session as T-view input. Subsequently, it extracts byte features and time-series features by utilizing transformer and BiLSTM modules.
- **DM-HNN** [22], similar to PEAN, is also an outstanding multi-view scheme in recent years. This scheme uses the header bytes and packet length sequences from the first 20 packets of each session as inputs to B-view and T-view. Then, it extracts features by using SAE and GRU modules.
- **PETNet** [23] is a state-of-the-art byte-based multi-view scheme. It uses the first 512 bytes from the first 16 packets of each session as input with a meta-feature extractor. A transformer encoder is used for extracting time-series features from TLS handshake packets and data packet byte sequences.
- **GLADS** [24] is a multi-view and multi-task ETC model combining depthwise separable convolution and global-local attention. The first

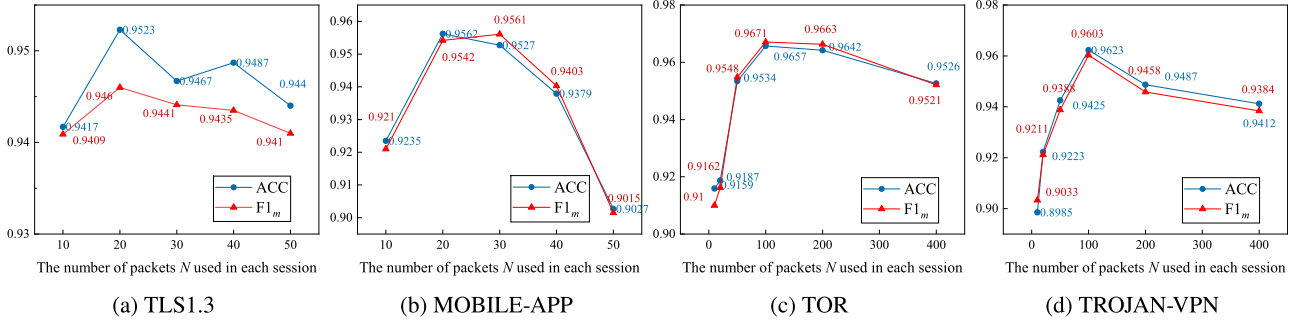


Fig. 7. ByteDance's classification performance changes with the values of N .

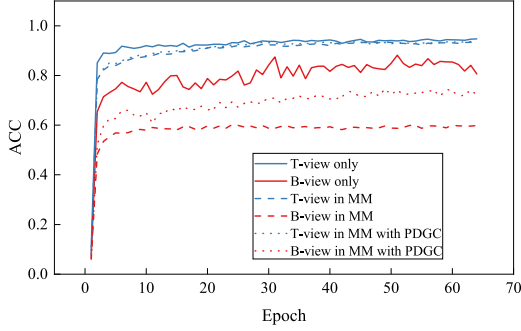


Fig. 8. Suppression of each view in ByteDance.

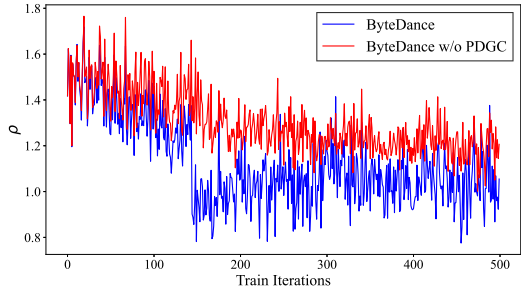


Fig. 9. Gradient ratio during train iterations.

32 packets' four header features and the first 784-byte segment of the transport layer payload are used as input.

5.4. Evaluation metrics

To evaluate ETC schemes, we use the following metrics: accuracy (ACC), true positive rate (TPR), false positive rate (FPR), macro-averaged F1 score ($F1_m$), and a fraction of TPR and FPR (FTF). Widely adopted in deep learning tasks [3,6], these metrics provide a comprehensive assessment of both overall performance and class-specific behavior.

Accuracy (ACC) is defined as:

$$ACC = \frac{TP + TN}{TP + TN + FP + FN} \quad (21)$$

where TP , TN , FP , and FN represent true positives, true negatives, false positives, and false negatives, respectively.

True positive rate (TPR), also referred to as recall, measures the proportion of actual positive samples correctly identified:

$$TPR_i = \frac{TP_i}{TP_i + FN_i} \quad (22)$$

False positive rate (FPR) quantifies the rate at which samples from other classes are incorrectly classified as class i :

$$FPR_i = \frac{FP_i}{FP_i + TN_i} \quad (23)$$

The macro-averaged F1 score ($F1_m$) calculates the average F1 score across all classes, making it robust for imbalanced datasets. For a specific class i , the F1 score is given as:

$$F1_i = 2 \times \frac{Precision_i \times Recall_i}{Precision_i + Recall_i} \quad (24)$$

where $Precision_i = \frac{TP_i}{TP_i + FP_i}$ and $Recall_i = \frac{TP_i}{TP_i + FN_i}$. The macro-averaged F1 score is then defined as:

$$F1_m = \frac{1}{N} \sum_{i=1}^N F1_i \quad (25)$$

Finally, the fractional combination of TPR and FPR (FTF) evaluates the overall classification performance by balancing positive detection capability (TPR) with the false positive rate (FPR). It is defined as:

$$FTF = \sum_{i=1}^N w_i \frac{TPR_i}{1 + FPR_i} \quad (26)$$

where w_i represents the weight of class i , reflecting the proportion of its samples to all flows. This metric emphasizes the adaptability of the model in handling imbalanced data by reducing the influence of high FPR values.

6. Performance evaluation

In this section, we conduct in-depth experiments aimed at providing a comprehensive response to the following research questions.

- **RQ1:** How to set the optimal values of N for different traffic datasets to achieve the best performance for ByteDance?
- **RQ2:** Can ByteDance classify encrypted traffic with promising classification performance?
- **RQ3:** Is the design of ByteDance's components effective?
- **RQ4:** Is ByteDance's operating efficiency during the training process efficient enough?

6.1. Hyperparameter experiments (RQ1)

In this subsection, we investigate the key hyperparameters N and M in ByteDance to optimize its classification performance. Specifically, M denotes the number of bytes utilized per packet in the B-view. Strictly speaking, M does not qualify as a hyperparameter, as its value is pre-determined through TBFE, which primarily selects plaintext bytes spanning from the network layer to the application layer headers, as detailed in Table 2. N represents the number of packets per session used during training and testing. The smaller the N value, the less data is used by the scheme, resulting in faster training and classification. The search

Table 5

Classification performance (%) comparison between ByteDance and other ETC schemes (TLS1.3 and TOR datasets). Results are in the format *avg. ($\pm$ std.)* obtained over 5-folds. **Bold** and underlined values indicate the best and second-best results, respectively.

Schemes	TLS1.3					TOR				
	ACC	TPR	FPR	FTF	F1 _m	ACC	TPR	FPR	FTF	F1 _m
FS-NET	88.91 $\pm$ 0.14	87.93 $\pm$ 0.13	0.23 $\pm$ 0.09	88.68 $\pm$ 0.13	87.91 $\pm$ 0.28	94.25 $\pm$ 0.16	94.14 $\pm$ 0.39	0.32 $\pm$ 0.08	94.03 $\pm$ 0.24	94.90 $\pm$ 0.32
RF	45.42 $\pm$ 0.14	45.60 $\pm$ 0.42	1.86 $\pm$ 0.21	46.03 $\pm$ 0.32	45.74 $\pm$ 0.26	<u>96.22$\pm$0.28</u>	<u>96.01$\pm$0.12</u>	0.32 $\pm$ 0.13	<u>96.14$\pm$0.36</u>	<u>96.39$\pm$0.56</u>
ET-BERT	<u>94.82$\pm$0.28</u>	95.41$\pm$0.21	0.20 $\pm$ 0.12	<u>94.54$\pm$0.34</u>	94.67$\pm$0.16	82.82 $\pm$ 0.28	83.63 $\pm$ 0.25	0.80 $\pm$ 0.46	81.73 $\pm$ 0.32	82.97 $\pm$ 0.35
TSCRNN	85.64 $\pm$ 0.12	82.34 $\pm$ 0.12	0.29 $\pm$ 0.08	<u>85.36$\pm$0.23</u>	83.45 $\pm$ 0.12	53.58 $\pm$ 0.24	53.26 $\pm$ 0.28	1.98 $\pm$ 0.48	53.01 $\pm$ 0.22	53.26 $\pm$ 0.18
MFFusion	87.57 $\pm$ 0.16	87.33 $\pm$ 0.33	0.25 $\pm$ 0.11	87.51 $\pm$ 0.15	87.74 $\pm$ 0.15	53.34 $\pm$ 0.26	53.32 $\pm$ 0.14	1.96 $\pm$ 0.56	52.42 $\pm$ 0.30	52.56 $\pm$ 0.10
DM-HNN	92.77 $\pm$ 0.22	91.37 $\pm$ 0.30	<u>0.15$\pm$0.05</u>	92.62 $\pm$ 0.31	91.73 $\pm$ 0.24	81.38 $\pm$ 0.12	81.26 $\pm$ 0.32	0.82 $\pm$ 0.17	81.18 $\pm$ 0.14	81.39 $\pm$ 0.33
PEAN	88.65 $\pm$ 0.36	86.66 $\pm$ 0.14	0.23 $\pm$ 0.06	88.43 $\pm$ 0.32	87.11 $\pm$ 0.35	39.46 $\pm$ 0.12	39.72 $\pm$ 0.28	2.57 $\pm$ 0.62	39.06 $\pm$ 0.15	38.30 $\pm$ 0.17
PETNet	92.09 $\pm$ 0.36	91.76 $\pm$ 0.27	0.17 $\pm$ 0.08	91.93 $\pm$ 0.31	92.11 $\pm$ 0.15	89.45 $\pm$ 0.12	90.47 $\pm$ 0.31	0.44 $\pm$ 0.08	88.83 $\pm$ 0.37	90.06 $\pm$ 0.37
GLADS	93.08 $\pm$ 0.35	89.83 $\pm$ 0.55	0.20 $\pm$ 0.01	92.94 $\pm$ 0.35	89.07 $\pm$ 0.55	89.52 $\pm$ 1.11	88.11 $\pm$ 1.84	0.46 $\pm$ 0.05	89.16 $\pm$ 1.14	86.56 $\pm$ 1.72
ByteDance	95.23$\pm$0.21	<u>94.82$\pm$0.15</u>	0.10$\pm$0.03	95.13$\pm$0.12	<u>94.60$\pm$0.13</u>	96.57$\pm$0.36	96.47$\pm$0.17	0.17$\pm$0.05	96.30$\pm$0.29	96.71$\pm$0.38

Table 6

Classification performance (%) comparison between ByteDance and other ETC schemes (TROJAN-VPN and MOBILE-APP datasets). Results are in the format *avg. ($\pm$ std.)* obtained over 5-folds. **Bold** and underlined values indicate the best and second-best results, respectively.

Schemes	TROJAN-VPN					MOBILE-APP				
	ACC	TPR	FPR	FTF	F1 _m	ACC	TPR	FPR	FTF	F1 _m
FS-NET	91.84 $\pm$ 0.12	91.91 $\pm$ 0.30	1.23 $\pm$ 0.26	91.77 $\pm$ 0.34	91.80 $\pm$ 0.29	89.53 $\pm$ 0.72	89.49 $\pm$ 0.43	1.50 $\pm$ 0.13	88.19 $\pm$ 0.37	89.40 $\pm$ 0.32
RF	57.39 $\pm$ 0.24	57.27 $\pm$ 0.39	2.10 $\pm$ 0.44	56.98 $\pm$ 0.62	57.11 $\pm$ 0.22	68.72 $\pm$ 0.14	69.07 $\pm$ 0.21	0.93 $\pm$ 0.34	68.80 $\pm$ 0.46	69.01 $\pm$ 0.31
ET-BERT	90.15 $\pm$ 0.25	89.07 $\pm$ 0.20	1.49 $\pm$ 0.38	90.11 $\pm$ 0.18	89.30 $\pm$ 0.31	85.27 $\pm$ 0.21	84.83 $\pm$ 0.19	1.72 $\pm$ 0.15	84.88 $\pm$ 0.26	84.76 $\pm$ 0.29
TSCRNN	66.32 $\pm$ 0.37	65.93 $\pm$ 0.24	4.89 $\pm$ 1.37	66.18 $\pm$ 0.28	66.26 $\pm$ 0.31	85.47 $\pm$ 0.29	84.73 $\pm$ 0.34	2.13 $\pm$ 0.87	85.27 $\pm$ 0.19	85.25 $\pm$ 0.21
MFFusion	62.47 $\pm$ 0.30	62.80 $\pm$ 0.37	5.38 $\pm$ 1.48	59.26 $\pm$ 0.38	61.98 $\pm$ 0.29	80.63 $\pm$ 0.43	81.33 $\pm$ 0.26	2.76 $\pm$ 0.47	78.53 $\pm$ 0.32	80.91 $\pm$ 0.21
DM-HNN	85.21 $\pm$ 0.24	84.70 $\pm$ 0.28	2.16 $\pm$ 0.51	84.56 $\pm$ 0.28	84.70 $\pm$ 0.37	87.21 $\pm$ 0.19	86.90 $\pm$ 0.32	1.77 $\pm$ 0.43	87.56 $\pm$ 0.17	87.13 $\pm$ 0.29
PEAN	80.94 $\pm$ 0.23	80.97 $\pm$ 0.31	2.79 $\pm$ 0.65	80.90 $\pm$ 0.36	81.12 $\pm$ 0.11	92.12 $\pm$ 0.23	92.23 $\pm$ 0.33	1.04 $\pm$ 0.17	91.28 $\pm$ 0.37	92.09 $\pm$ 0.14
PETNet	92.96 $\pm$ 0.28	92.58 $\pm$ 0.10	0.85 $\pm$ 0.09	92.89 $\pm$ 0.20	92.26 $\pm$ 0.35	90.05 $\pm$ 0.10	89.98 $\pm$ 0.23	1.42 $\pm$ 0.14	88.77 $\pm$ 0.31	89.85 $\pm$ 0.36
GLADS	86.41 $\pm$ 1.25	86.38 $\pm$ 1.37	1.94 $\pm$ 0.18	84.83 $\pm$ 1.37	85.62 $\pm$ 1.34	<u>93.05$\pm$0.31</u>	<u>92.95$\pm$0.30</u>	0.89 $\pm$ 0.04	92.19 $\pm$ 0.36	92.79 $\pm$ 0.32
ByteDance	96.23$\pm$0.28	96.31$\pm$0.12	0.51$\pm$0.06	96.29$\pm$0.16	96.03$\pm$0.35	95.62$\pm$0.19	95.72$\pm$0.21	0.63$\pm$0.03	95.49$\pm$0.23	95.42$\pm$0.26

range for N is determined by the distribution of the packet numbers in each dataset. Specifically, for the TLS1.3 dataset, N is varied from 10 to 50; for the Tor and TROJAN-VPN datasets, the range is 10 to 400. Although the MOBILE-APP dataset typically contains sessions with many more packets, experiments show that classification performance drops sharply when N exceeds 30. Consequently, we only show results when N is set from 10 to 50 on this dataset. The experimental results are shown in Fig. 7, where the horizontal axis represents different N values. The following conclusions can be drawn.

On the TLS1.3 dataset, ByteDance achieves the highest accuracy and F1_m of 95.23% and 94.60% when N equals 20. On the MOBILE-APP dataset, when N is set to 20 and 30, our model shows the best performance. On the TOR dataset, the scheme achieves the highest accuracy and F1_m of 96.57% and 96.71% when N equals 100. On the TROJAN-VPN dataset, ByteDance achieves the highest accuracy of 96.23% and F1_m of 96.03% when N is set to 100. Based on the experimental findings, we determined that $N = 20$ is suitable for the TLS1.3 and MOBILE-APP datasets, and $N = 100$ is appropriate for the TOR and TROJAN-VPN datasets.

Despite ByteDance's top performance on the last two datasets with higher N values, it maintains good classification performance even when N is set to 20. When N exceeds the optimal value selected above, classification performance generally decreases, indicating that using too many packets does not improve classification ability. Instead, it may increase training time and reduce classification accuracy. This demonstrates that our scheme can achieve strong classification performance even when using fewer packets.

6.2. Comparative experiments (RQ2)

In this subsection, ByteDance is compared with nine state-of-the-art ETC schemes. During our experiments, we evaluate all schemes with their optimal hyperparameters, using the comparative approach com-

mon in previous studies [7,9,12,16,20–24]. The complete experimental results are shown in Tables 5 and 6. The following conclusions can be drawn.

ByteDance achieves the best performance on four different encrypted traffic datasets. First, it outperforms on all four datasets, achieving the highest accuracy and exceeding the optimal baseline by at least 0.35% and up to 3.27%. Second, ByteDance shows significant improvements in other metrics, demonstrating better classification balance across all traffic categories compared to other schemes. It achieves the highest TPR on three of the four datasets, only slightly lower than ET-BERT on the TLS1.3 dataset. ByteDance obtains the lowest FPR values in all four datasets, with differences of 0.05% to 0.34% from the nearest competitor. As shown in Tables 5 and 6, ByteDance's FTF scores exceed 95% on all four datasets, surpassing all baseline schemes. Its F1_m scores are the highest on three datasets, only falling behind ET-BERT by 0.07% on the TLS1.3 dataset. ByteDance's performance advantage stems from its unique design. In contrast to previous schemes that simply treat byte sequences as either text or images for feature extraction, ByteDance introduces an innovative TBFE strategy. Initially, it discards a multitude of encrypted and irrelevant byte sequences by employing fuzzy truncation. Then, using intra-packet and inter-packet byte feature extraction modules, it accurately extracts useful header byte information from the protocol stack. Moreover, ByteDance tackles the view suppression problem by improving B-view with the PDGC strategy, ensuring that T-view stays unaffected. This will be verified in subsequent ablation experiments.

To eliminate biases from inconsistent packet numbers, we apply a uniform input packet count across all ETC schemes and analyze their classification performance. Tables 7 and 8 show the results when the number of input packets is set to 10 and 20, respectively. Since ET-BERT recommends packet-level classification, while it supports flow-level input, its performance degrades significantly. Therefore, we exclude it from the comparison in this experiment.

Table 7

Classification performance (%) with varying numbers of packets (TLS1.3 and TOR datasets). Results are in the format *avg. ($\pm$ std.)* obtained over 5-folds. **Bold** and underlined values indicate the best and second-best results, respectively.

Schemes	TLS1.3				TOR			
	10 packets		20 packets		10 packets		20 packets	
	ACC	F1 _m	ACC	F1 _m	ACC	F1 _m	ACC	F1 _m
FS-NET	81.23 $\pm$ 0.23	81.82 $\pm$ 0.18	86.23 $\pm$ 0.38	86.42 $\pm$ 0.21	68.97 $\pm$ 0.70	68.85 $\pm$ 0.72	74.02 $\pm$ 0.65	74.00 $\pm$ 0.64
RF	39.92 $\pm$ 0.27	39.84 $\pm$ 0.09	42.35 $\pm$ 0.26	42.14 $\pm$ 0.24	79.26 $\pm$ 0.37	80.16 $\pm$ 0.14	86.22 $\pm$ 0.46	86.18 $\pm$ 0.45
TSCRNN	85.34 $\pm$ 0.12	85.25 $\pm$ 0.12	88.53 $\pm$ 0.28	88.48 $\pm$ 0.35	53.58 $\pm$ 0.24	53.26 $\pm$ 0.18	63.71 $\pm$ 0.78	63.77 $\pm$ 0.79
MMFusion	84.42 $\pm$ 0.33	84.21 $\pm$ 0.17	85.32 $\pm$ 0.34	85.28 $\pm$ 0.12	49.07 $\pm$ 0.90	49.03 $\pm$ 0.91	72.27 $\pm$ 0.68	71.57 $\pm$ 0.69
DM-HNN	89.79 $\pm$ 0.23	89.67 $\pm$ 0.23	92.77 $\pm$ 0.22	91.73 $\pm$ 0.24	74.40 $\pm$ 0.63	74.48 $\pm$ 0.65	81.38 $\pm$ 0.12	81.39 $\pm$ 0.33
PEAN	88.65 $\pm$ 0.36	87.11 $\pm$ 0.35	87.35 $\pm$ 0.20	86.91 $\pm$ 0.32	39.46 $\pm$ 0.12	38.03 $\pm$ 0.17	41.43 $\pm$ 1.05	41.40 $\pm$ 1.06
PETNet	90.37 $\pm$ 0.37	90.15 $\pm$ 0.20	92.87 $\pm$ 0.42	91.71 $\pm$ 0.17	85.81 $\pm$ 0.47	85.82 $\pm$ 0.48	88.23 $\pm$ 0.43	88.21 $\pm$ 0.44
GLADS	91.12 $\pm$ 0.58	86.40 $\pm$ 0.94	92.23 $\pm$ 0.51	87.99 $\pm$ 0.77	62.15 $\pm$ 1.50	57.16 $\pm$ 1.99	80.99 $\pm$ 1.08	76.57 $\pm$ 1.52
ByteDance	94.17$\pm$0.33	94.09$\pm$0.15	95.23$\pm$0.21	94.60$\pm$0.13	91.59$\pm$0.42	91.00$\pm$0.44	91.87$\pm$0.38	91.62$\pm$0.39

Table 8

Classification performance (%) with varying numbers of packets (TROJAN-VPN and MOBILE-APP datasets). Results are in the format *avg. ($\pm$ std.)* obtained over 5-folds. **Bold** and underlined values indicate the best and second-best results, respectively.

Schemes	TROJAN-VPN				MOBILE-APP			
	10 packets		20 packets		10 packets		20 packets	
	ACC	F1 _m	ACC	F1 _m	ACC	F1 _m	ACC	F1 _m
FS-NET	71.39 $\pm$ 0.68	71.30 $\pm$ 0.69	81.29 $\pm$ 0.58	81.05 $\pm$ 0.59	79.85 $\pm$ 0.60	80.10 $\pm$ 0.61	87.00 $\pm$ 0.50	86.80 $\pm$ 0.51
RF	48.34 $\pm$ 0.29	47.64 $\pm$ 0.32	52.19 $\pm$ 0.27	52.23 $\pm$ 0.17	57.22 $\pm$ 0.31	57.46 $\pm$ 0.30	60.52 $\pm$ 0.26	60.48 $\pm$ 0.14
TSCRNN	66.32 $\pm$ 0.37	66.26 $\pm$ 0.31	75.01 $\pm$ 0.66	74.87 $\pm$ 0.67	85.47 $\pm$ 0.29	85.25 $\pm$ 0.21	78.60 $\pm$ 0.60	78.30 $\pm$ 0.61
MMFusion	57.34 $\pm$ 0.85	57.32 $\pm$ 0.86	63.91 $\pm$ 0.76	63.67 $\pm$ 0.77	65.00 $\pm$ 0.75	65.10 $\pm$ 0.76	70.20 $\pm$ 0.65	69.90 $\pm$ 0.66
DM-HNN	74.91 $\pm$ 0.62	74.98 $\pm$ 0.63	85.21 $\pm$ 0.24	84.70 $\pm$ 0.37	83.10 $\pm$ 0.55	83.30 $\pm$ 0.56	87.21 $\pm$ 0.19	87.13 $\pm$ 0.29
PEAN	80.94 $\pm$ 0.23	81.12 $\pm$ 0.11	82.32 $\pm$ 0.55	82.49 $\pm$ 0.56	92.12 $\pm$ 0.23	92.09 $\pm$ 0.14	87.80 $\pm$ 0.48	87.90 $\pm$ 0.49
PETNet	89.67 $\pm$ 0.48	90.81$\pm$0.47	<u>91.12$\pm$0.42</u>	<u>90.89$\pm$0.43</u>	91.50 $\pm$ 0.35	91.60 $\pm$ 0.36	<u>93.20$\pm$0.28</u>	<u>93.00$\pm$0.29</u>
GLADS	78.85 $\pm$ 1.42	77.66 $\pm$ 1.64	84.50 $\pm$ 1.12	83.15 $\pm$ 1.10	92.62$\pm$0.14	92.54$\pm$0.07	92.93 $\pm$ 0.27	92.73 $\pm$ 0.24
ByteDance	89.85$\pm$0.47	<u>90.33$\pm$0.46</u>	92.23$\pm$0.40	92.11$\pm$0.41	<u>92.35$\pm$0.30</u>	<u>92.10$\pm$0.32</u>	95.62$\pm$0.19	95.42$\pm$0.26

The results in [Tables 7 and 8](#) demonstrate that ByteDance's classification performance remains strong with a consistent number of input packets. When set to 20 input packets, its accuracy and F1_m score surpass all baselines on all four datasets. When set to 10 input packets, ByteDance outperforms other baseline schemes on the TLS1.3 and TOR datasets. On the TROJAN-VPN dataset, ByteDance achieves the highest accuracy and the second-best F1_m score. While its performance on the MOBILE-APP dataset is slightly lower than GLADS, it still surpasses other baseline schemes. It is important to note that schemes like FS-NET, which performed well on the TOR and TROJAN-VPN datasets in previous experiments, face a notable decline in classification accuracy as the number of packets decreases. This is because such schemes overly rely on the distributional features provided by long packet sequences.

6.3. Ablation experiments (RQ3)

To gain a deep understanding of the efficacy and rationality of the essential modules employed by ByteDance, we have designed and performed a set of ablation experiments on the TLS1.3 dataset, with the input packet count fixed at 20. By comparing the performance of the following different variants, we explore how each module contributes to the overall scheme.

- w/o **B-view**: Remove the B-view and retain only the T-view, showing that the TBFE strategy is not applied to highlight its effectiveness.
- w/o **T-view**: Remove the T-view and retain only the B-view to demonstrate the effectiveness of the T-view.
- w/o **PDGC**: Remove the PDGC strategy to assess its effectiveness in mitigating the B-view suppression by the T-view.
- w/o **TBFE-nFBS**: Remove the fuzzy byte selection from the first stage of TBFE and instead use the first 400 bytes of each packet (a standard

Table 9

Ablation study (%) on the TLS 1.3 dataset. Results are in the format *avg. ($\pm$ std.)* obtained over 5-folds.

Schemes	ACC	TPR	FPR	FTF	F1 _m
ByteDance	95.23 $\pm$ 0.21	94.82 $\pm$ 0.15	0.10 $\pm$ 0.03	95.13 $\pm$ 0.12	94.60 $\pm$ 0.13
w/o B-view	92.73 $\pm$ 0.13	92.49 $\pm$ 0.14	0.15 $\pm$ 0.03	92.24 $\pm$ 0.22	92.25 $\pm$ 0.11
w/o T-view	81.52 $\pm$ 0.19	81.33 $\pm$ 0.12	0.89 $\pm$ 0.08	81.17 $\pm$ 0.32	80.72 $\pm$ 0.25
w/o PDGC	93.03 $\pm$ 0.42	92.92 $\pm$ 0.17	0.14 $\pm$ 0.01	93.15 $\pm$ 0.25	93.22 $\pm$ 0.13
w/o TBFE-nFBS	93.45 $\pm$ 0.15	93.43 $\pm$ 0.17	0.14 $\pm$ 0.05	93.55 $\pm$ 0.15	93.15 $\pm$ 0.11
w/o TBFE-nLB	89.74 $\pm$ 0.21	89.25 $\pm$ 0.15	0.57 $\pm$ 0.08	89.44 $\pm$ 0.23	89.18 $\pm$ 0.28
w/o TBFE-nGB	85.18 $\pm$ 0.39	85.56 $\pm$ 0.24	0.70 $\pm$ 0.09	85.43 $\pm$ 0.36	85.68 $\pm$ 0.16

length used in previous ETC schemes [21,25]) to assess the importance of fuzzy byte selection.

- w/o **TBFE-nLB**: Remove the local byte feature extraction module from the precise feature extraction component of TBFE. That is, the fixed-window local attention mechanism for modeling intra-packet byte sequences is no longer employed. Instead, the raw byte sequences are directly fed into the global feature extraction module for holistic representation learning.
- w/o **TBFE-nGB**: Remove the global byte feature extraction module from the precise feature extraction component of TBFE. That is, the transformer encoder is no longer used to integrate the intra-packet features from the local attention mechanism. Instead, the extracted features from individual packets are used directly as the final representation of the feature extraction stage.

The experimental results are shown in [Table 9](#), and the following conclusions can be drawn.

Firstly, we verify the effectiveness of B-view and T-view in ByteDance. When B-view is removed (w/o **B-view**), the scheme's

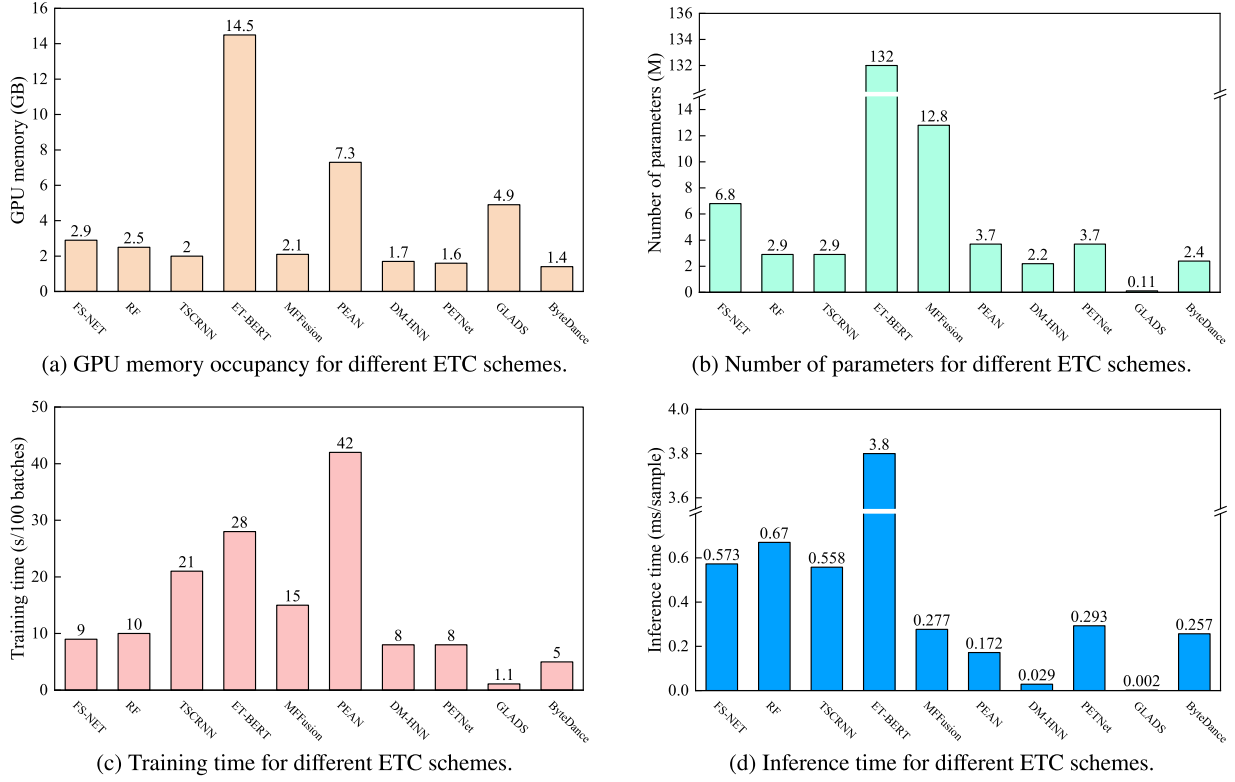


Fig. 10. Operating efficiency comparison between ByteDance and other ETC schemes.

accuracy with only the T-view is 92.73 %. TPR and $F1_m$ drop to 92.49 % and 92.25 %, and FPR rises to 0.15 %. These results show that the TBFE strategy is vital for better classification accuracy and low false positive rates, confirming the significance of byte features in ByteDance. When T-view is removed (w/o **T-view**), the accuracy falls to 81.52 %, TPR to 81.33 %, and FPR rises to 0.89 %. Besides, FTF and F1 decrease to 81.17 % and 80.72 %, respectively. The significant performance drop highlights T-view's role in tracking packet sequence changes and boosting ByteDance's robustness.

Secondly, we examine how effective the PDGC strategy is in mitigating the suppression of B-view. When the PDGC strategy is removed (w/o **PDGC**), the accuracy drops to 93.03 %, TPR falls to 92.92 %, and FPR rises to 0.14 %. These results demonstrate that the PDGC strategy alleviates the suppression of B-view by T-view and maintains key advantages for balanced multi-view feature representation.

Thirdly, we assess the effectiveness of fuzzy byte selection used in the first stage of TBFE. When fuzzy byte selection is removed from TBFE (w/o **TBFE-nFBS**), the accuracy decreases to 93.45 % by utilizing the first 400 bytes of each packet as input. TPR measures 93.43 %, with FPR rising to 0.14 %. Furthermore, the FTF and $F1_m$ values decrease to 93.55 % and 93.15 %, respectively. The results demonstrate that the byte fuzzy selection mechanism effectively filters out irrelevant bytes and improves the focus on key byte features, enhancing overall performance.

Finally, we examine the role of precise byte feature extraction carried out by TBFE. Specifically, we independently ablate the local byte feature extractor (w/o **TBFE-nLB**) and the global byte feature extractor (w/o **TBFE-nGB**). As shown in Table 9, the classification performance of both variants degrades significantly compared with full ByteDance. The absence of local byte feature extraction results in an accuracy decrease of around 6 %, whereas removing global byte feature extraction leads to a significant accuracy decline of approximately 10 %. These results demonstrate that both local and global byte feature extraction modules are essential components of TBFE.

To thoroughly analyze the impact of the PDGC strategy on mitigating the view suppression dilemma, we illustrate ByteDance's accuracy curves in three contexts: a single-view model, a multi-view model, and a multi-view model enhanced by the PDGC strategy. In Fig. 8, the solid line represents accuracy from T-view or B-view alone; the dashed line shows accuracy with T-view and B-view in a multi-view model; the dotted line indicates accuracy from T-view and B-view in a multi-view model with PDGC. Fig. 8 indicates that the PDGC strategy mitigates the suppression of T-view on B-view without negatively affecting the dominant T-view. To assess the effectiveness of the PDGC strategy in balancing gradients, we construct a plot displaying the gradient ratio between the T-view and B-view throughout the training process, following Eq. (15). As illustrated in Fig. 9, full ByteDance maintains a lower T-view/B-view gradient ratio during training in contrast to its variant that removes the PDGC strategy. This provides an intuitive illustration of the PDGC's significant role in promoting gradient balance across multiple views.

6.4. Operating efficiency experiments (RQ4)

In this subsection, we conduct an overhead analysis of ByteDance and all the baselines, including parameter counts, GPU memory occupancy during training, training time (s/100 batch), and inference time (ms/sample). To ensure fairness, only one CPU or GPU task runs at a time, with a batch size of 128 for both training and testing.

Fig. 10 illustrates the GPU memory occupancy of each scheme during training. ByteDance uses only 1472 MB of GPU memory for training, making it the most memory-efficient scheme. The main reason is ByteDance's smaller embedding dimension in T-view and its use of a single transformer encoder for extracting features. In addition, the B-view leverages fuzzy byte selection in TBFE, ensuring that only a minimal number of useful bytes are processed. ByteDance employs local attention for each packet, but the local attention module shares a single set of parameters, which saves memory. The schemes with pre-trained

models such as PEAN require additional GPU memory since they learn attention embeddings from the first 400 bytes of packets, leading to intensive computations and many intermediate variables.

The remaining three figures depict the parameter counts, training time, and inference time for all ETC schemes, respectively. GLADS has the lowest values for these metrics but uses 4.9 GB of GPU memory and performs worse than ByteDance on all four datasets. ET-BERT matches ByteDance in performance on the TLS 1.3 dataset, but it uses significantly more computational resources and memory than other schemes. Besides, ByteDance has more parameters than DM-HNN and takes longer to infer than both DM-HNN and PEAN, but it provides much better classification accuracy across all four datasets. Therefore, these results demonstrate that ByteDance achieved a better balance between classification performance and runtime efficiency.

7. Conclusion

In this paper, we conduct a pioneering study on the dilemma that multi-view ETC schemes cannot perform as expected. Firstly, we discover that inaccurate byte feature extraction and B-view suppression during scheme training are the two main problems in current research through comparative experiments. Secondly, we propose ByteDance, a groundbreaking approach enabling bytes to perform brilliantly in multi-view ETC. To accurately capture byte features, a two-stage byte feature extraction strategy (TBFE) was designed. It first screens packet bytes based on their protocol format and byte distribution, then employs local attention module and transformer encoder to accurately extract byte features. To mitigate the suppression of B-view, a prototype-network-based dynamic gradient compensation strategy (PDGC) is proposed. In every training batch, PDGC assigns weights to the gradients of multiple views, enhancing the strength of gradient propagation in the B-view using PCE loss function. Finally, we conduct comprehensive experiments to thoroughly validate ByteDance. It achieves remarkable classification accuracy on four distinct network traffic datasets, significantly surpassing all baselines. While sustaining high-level classification performance, ByteDance simultaneously exhibits comparatively superior operational efficiency relative to the majority of baselines. The ablation experiment substantiates the effectiveness of the two strategies, TBFE and PDGC, in model classification.

In summary, ByteDance overcomes the shortcomings of existing research and fully leverages the advantages of multi-view learning in ETC. In the future, we will focus on adding more views to the multi-view ETC method to enhance its generalization and performance in complex classifications.

CRedit authorship contribution statement

Yuwei Xu: Validation, Supervision, Resources, Methodology, Funding acquisition, Formal analysis, Conceptualization; **Zhiyuan Liang:** Writing – original draft, Methodology, Investigation, Formal analysis; **Xiaotian Fang:** Writing – review & editing, Validation, Software, Data curation; **Kehui Song:** Writing – review & editing, Validation, Formal analysis, Conceptualization; **Meng Wang:** Writing – review & editing, Validation, Resources; **Qiao Xiang:** Writing – review & editing, Supervision, Resources, Conceptualization; **Guang Cheng:** Writing – review & editing, Resources, Funding acquisition.

Data availability

We plan to share the code and data of ByDance on GitHub after this paper is officially published.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported in part by the CCF-NSFOCUS 'Kunpeng' Research Fund under Grant No. CCF-NSFOCUS 2024010, in part by the National Key R&D Program of China under Grant No. 2023YFB3106700, and in part by the [National Natural Science Foundation of China](#) under Grant Nos. [62172093](#) and [U22B2025](#).

References

- [1] G. Report, Google transparency report, 2025, (URL <https://transparencyreport.google.com/>).
- [2] S. Rezaei, X. Liu, Deep learning for encrypted traffic classification: an overview, *IEEE Commun. Mag.* 57 (5) (2019) 76–81.
- [3] M. Shen, K. Ye, X. Liu, et al., Machine learning-powered encrypted network traffic analysis: a comprehensive survey, *IEEE Commun. Surv. Tutor.* 25 (1) (2022) 791–824.
- [4] C. Chen, Y. Wu, Q. Dai, et al., A survey on graph neural networks and graph transformers in computer vision: a task-Oriented perspective, *IEEE Trans. Pattern Anal. Mach. Intell.* 46 (12) (2024) 10297–10318.
- [5] D. Khurana, A. Koli, K. Khatter, et al., Natural language processing: state of the art, current trends and challenges, *Multimed Tools Appl.* 82 (3) (2023) 3713–3744.
- [6] A. Sharma, A.H. Lashkari, A survey on encrypted network traffic: a comprehensive survey of identification/classification techniques, challenges, and future directions, *Comput. Netw.* 257 (2024) 110984–111011.
- [7] C. Liu, L. He, G. Xiong, et al., Fs-net: a flow sequence network for encrypted traffic classification, in: *Proceedings of the 38th IEEE Conference on Computer Communications (INFOCOM)*, 2019, pp. 1171–1179.
- [8] P. Sirinam, M. Imani, M. Juarez, M. Wright, Deep fingerprinting: undermining website fingerprinting defenses with deep learning, in: *Proceedings of the 25th ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018, pp. 1928–1943.
- [9] M. Shen, K. Ji, Z. Gao, et al., Subverting website fingerprinting defenses with robust traffic representation, in: *Proceedings of the 32nd USENIX Security Symposium (USENIX Security)*, USENIX Association, Anaheim, CA, 2023, pp. 607–624.
- [10] Q. Zhou, L. Wang, H. Zhu, et al., WF-Transformer: learning temporal features for accurate anonymous traffic identification by using transformer networks, *IEEE Trans. Inf. Forensics Secur.* 19 (2024) 30–43.
- [11] J. Qu, X. Ma, J. Li, et al., An input-agnostic hierarchical deep learning framework for traffic fingerprinting, in: *Proceedings of the 32nd USENIX Security Symposium (USENIX Security)*, USENIX Association, Anaheim, CA, 2023, pp. 589–606.
- [12] X. Lin, G. Xiong, G. Gou, et al., ET-BERT: a contextualized datagram representation with pre-training transformers for encrypted traffic classification, in: *Proceedings of the 31st ACM Web Conference (WWW)*, 2022, pp. 633–642.
- [13] J. Yu, Y. Choi, K. Koo, D. Moon, A novel approach for application classification with encrypted traffic using BERT and packet headers, *Comput. Netw.* 254 (2024) 110747–110758.
- [14] M. Zhan, J. Yang, D. Jia, G. Fu, EAPT: an encrypted traffic classification model via adversarial pre-trained transformers, *Comput. Netw.* 257 (2025) 110973–110981.
- [15] W. Wang, M. Zhu, J. Wang, et al., End-to-end encrypted traffic classification with one-dimensional convolution neural networks, in: *Proceedings of the 15th IEEE International Conference on Intelligence and Security Informatics (ISI)*, 2017, pp. 43–48.
- [16] K. Lin, X. Xu, H. Gao, TSCRNN: a novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT, *Comput. Netw.* 190 (2021) 107974–107984.
- [17] G. Xie, Q. Li, Y. Jiang, Self-attentive deep learning method for online traffic classification and its interpretability, *Comput. Netw.* 196 (2021) 108267–108278.
- [18] X. Ma, W. Zhu, J. Wei, et al., EETC: an extended encrypted traffic classification algorithm based on variant resnet network, *Comput. Secur.* 128 (2023) 103175–103187.
- [19] X. Wang, S. Chen, J. Su, App-net: a hybrid neural network for encrypted mobile traffic classification, in: *Proceedings of the 39th IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, 2020, pp. 424–429.
- [20] K. Lin, X. Xu, F. Xiao, MFFusion: A multi-level features fusion model for malicious traffic detection based on deep learning, *Comput. Netw.* 202 (2022) 108658–108671.
- [21] P. Lin, K. Ye, Y. Hu, et al., A novel multimodal deep learning framework for encrypted traffic classification, *IEEE/ACM Trans. Networking* 31 (3) (2023) 1369–1384.
- [22] Y. Yang, Y. Yan, Z. Gao, et al., A network traffic classification method based on dual-mode feature extraction and hybrid neural networks, *IEEE Trans. Netw. Serv. Manage.* 20 (4) (2023) 4073–4084.
- [23] X. Yang, S. Ruan, Y. Yue, B. Sun, PETNet: plaintext-aware encrypted traffic detection network for identifying cobalt strike HTTPS traffics, *Comput. Netw.* 238 (2024) 110120–110132.
- [24] J. Dai, X. Xu, F. Xiao, GLADS: a global-local attention data selection model for multimodal multitask encrypted traffic classification of IoT, *Comput. Netw.* 225 (2023) 109652–109669.
- [25] X. Han, G. Xu, M. Zhang, et al., DE-GNN: dual embedding with graph neural network for fine-grained encrypted traffic classification, *Comput. Netw.* 245 (2024) 110372–110389.

- [26] G. Aceto, D. Ciunzono, A. Montieri, A. Pescapè, MIMETIC: mobile encrypted traffic classification using multimodal deep learning, *Comput. Netw.* 165 (2019) 106944–106955.
- [27] M.A.I.M. Aminuddin, Z.F. Zaaba, A. Samsudin, et al., The rise of website fingerprinting on tor: analysis on techniques and assumptions, *J. Netw. Comput. Appl.* 212 (2023) 103582–103608.
- [28] W. Dong, J. Yu, X. Lin, et al., Deep learning and pre-training technology for encrypted traffic classification: a comprehensive review, *Neurocomputing* (2024) 128444–128469.
- [29] X. Peng, Y. Wei, A. Deng, et al., Balanced multimodal learning via on-the-fly gradient modulation, in: *Proceedings of the 39th IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 8238–8247.
- [30] Y. Fan, W. Xu, H. Wang, et al., Pmr: prototypical modal rebalance for multimodal learning, in: *Proceedings of the 40th IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 20029–20038.
- [31] R. Fu, Z. Zhang, L. Li, Using LSTM and GRU neural network methods for traffic flow prediction, in: *Proceedings of the 31st Youth Academic Annual Conference of Chinese Association of Automation (YAC)*, 2016, pp. 324–328.
- [32] A. Vaswani, N. Shazeer, N. Parmar, et al., Attention is all you need, in: *Proceedings of the 31st Conference on Neural Information Processing Systems (NIPS)*, 2017, pp. 6000–6010.
- [33] J. Devlin, M.W. Chang, K. Lee, K. Toutanova, Bert: pre-training of deep bidirectional transformers for language understanding, in: *Proceedings of the 17th Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2019, pp. 4171–4186.
- [34] Y. Xu, J. Cao, K. Song, et al., Fasttraffic: a lightweight method for encrypted traffic fast classification, *Comput. Netw.* 235 (2023) 51–63.
- [35] Z. Chen, V. Badrinarayanan, C.Y. Lee, A. Rabinovich, Gradnorm: gradient normalization for adaptive loss balancing in deep multitask networks, in: *Proceedings of the 35th International Conference on Machine Learning, PMLR*, 2018, pp. 794–803.
- [36] T. Yu, S. Kumar, A. Gupta, et al., Gradient surgery for multi-task learning, *Adv. Neural Inf. Process. Syst.* 33 (2020) 5824–5836.
- [37] S. Wire, wireshark official website, 2025, (URL <https://www.wireshark.org/>).
- [38] José Armando Tesén Marañón, Romaric Duvignau, 5G Mobile App Traffic Traces (Chalmers 2023), IEEE Dataport, 2024, <https://dx.doi.org/10.21227/acn1-ek95>.
- [39] J. Gong, W. Zhang, C. Zhang, T. Wang, Surakav: generating realistic traces for a strong website fingerprinting defense, in: *Proceedings of the 43rd IEEE Symposium on Security and Privacy (S&P)*, 2022, pp. 1558–1573.
- [40] C. Li, L. Nie, L. Zhao, K. Li, Robust website fingerprinting through resource loading sequence, *World Wide Web: Internet and Web Information Systems* 26 (5) (2023) 2329–2349.
- [41] V. Project, v2ray official website, 2025, (URL <https://www.v2ray.com/>).
- [42] Y. Xu, X. Fang, J. Cao, et al., Cerberus: efficient OSPS traffic identification through multi-task learning, in: *Proceedings of the 22nd IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2023, pp. 504–511.